

НАЦІОНАЛЬНИЙ ТЕХНІЧНИЙ УНІВЕРСИТЕТ УКРАЇНИ
«КИЇВСЬКИЙ ПОЛІТЕХНІЧНИЙ ІНСТИТУТ імені ІГОРЯ СІКОРСЬКОГО»
Факультет інформатики та обчислювальної техніки
(повна назва інституту/факультету)

Кафедра інформатики та програмної інженерії
(повна назва кафедри)

«До захисту допущено»

Завідувач кафедри

_____ Едуард ЖАРІКОВ
(підпис) (ім'я прізвище)

“ ____ ” _____ 2022 р.

Дипломний проєкт
на здобуття ступеня бакалавра
за освітньо-професійною програмою «Інженерія програмного забезпечення
комп'ютеризованих систем»
спеціальності «121 Інженерія програмного забезпечення»

на тему: Програмне забезпечення для підтримки та аналітики для продавців
на торговельних майданчиках

Виконав студент IV курсу, групи ІП-82
(шифр групи)

Філатов Олександр Олександрович _____
(прізвище, ім'я, по батькові) (підпис)

Керівник доцент, к.т.н., доц., Фіногенов О. Д. _____
(посада, науковий ступінь, вчене звання, прізвище та ініціали) (підпис)

Консультант
з графічної
документації доцент, к.т.н., доц., Ліщук К. І. _____
(посада, науковий ступінь, вчене звання, прізвище та ініціали) (підпис)

Рецензент д.т.н., доц., професор кафедри ІСТ Ярослав
КОРНАГА _____
(посада, науковий ступінь, вчене звання, прізвище та ініціали) (підпис)

Засвідчую, що у цій дипломній роботі
немає запозичень з праць інших
авторів без відповідних посилань.

Студент _____
(підпис)

Київ – 2022

Національний технічний університет України
«Київський політехнічний інститут імені Ігоря Сікорського»

Факультет інформатики та обчислювальної техніки

Кафедра інформатики та програмної інженерії

Рівень вищої освіти – перший (бакалаврський)

Спеціальність – 121 Інженерія програмного забезпечення

Освітньо-професійна програма – Інженерія програмного забезпечення
комп'ютеризованих систем

ЗАТВЕРДЖУЮ

Завідувач кафедри

_____ Едуард ЖАРІКОВ

(підпис)

(ім'я прізвище)

“ ____ ” _____ 2022 р.

ЗАВДАННЯ

на дипломний проєкт студенту

Філатову Олександру Олександровичу

(прізвище, ім'я, по батькові)

1. Тема проєкту Програмне забезпечення для підтримки та аналітики для
продавців на торговельних майданчиках

керівник проєкту Фіногенов О. Д.

(прізвище, ім'я, по батькові, науковий ступінь, вчене звання)

затверджені наказом по університету від «10» червня 2022 р. № 1033-с

2. Термін подання студентом проєкту «19» червня 2022 року

3. Вихідні дані до проєкту: технічне завдання

4. Зміст пояснювальної записки

1) Аналіз вимог до програмного забезпечення: основні визначення та терміни, опис предметного середовища, огляд існуючих рішень, постановка задачі.

2) Моделювання та конструювання програмного забезпечення: моделювання та аналіз програмного забезпечення, засоби розробки, технічні рішення, архітектура програмного забезпечення.

3) Аналіз якості та тестування програмного забезпечення.

4) Розгортання та впровадження програмного забезпечення

5. Перелік графічного матеріалу

1) Схема структурна варіантів використання.

2) Схема структурна контексту програмного забезпечення.

3) Схема структурна контейнерів програмного забезпечення.

4) Схема структурна компонентів програмного забезпечення.

6. Консультанти розділів проєкту

Розділ	Прізвище, ініціали та посада консультанта	Підпис, дата	
		завдання видав	завдання прийняв

7. Дата видачі завдання: «10» березня 2022 року

Календарний план

№ з/п	Назва етапів виконання дипломного проєкту	Термін виконання етапів проєкту	Примітка
1	Вивчення рекомендованої літератури	10.03.2022-14.03.2022	
2	Аналіз існуючих методів розв'язання задачі	15.03.2022-21.03.2022	
3	Постановка та формалізація задачі	22.03.2022-28.03.2022	
4	Розробка інформаційного забезпечення	29.03.2022-05.04.2022	
5	Алгоритмізація задачі	06.04.2022-12.04.2022	

6	Обґрунтування вибору використаних технічних засобів	13.04.2022-19.04.2022	
7	Розробка програмного забезпечення	20.04.2022-13.05.2022	
8	Налагодження програми	14.05.2022-19.05.2022	
9	Виконання графічних документів	20.05.2022-24.05.2022	
10	Оформлення пояснювальної записки	25.05.2022-05.06.2022	
11	Подання ДП на попередній захист	06.06.2022	
12	Подання ДП рецензенту	12.06.2022	
13	Подання ДП на основний захист	19.06.2022	

Студент

(підпис)

Олександр ФІЛАТОВ

(ініціали, прізвище)

Керівник

(підпис)

Олексій ФІНОГЕНОВ

(ініціали, прізвище)

АНОТАЦІЯ

Пояснювальна записка дипломного проєкту складається з чотирьох розділів, містить 20 таблиць, 18 рисунків та 15 джерел – загалом 56 сторінок.

Даний дипломний проєкт присвячений створенню програмного забезпечення для контролювання товарів одразу з декількох торговельних майданчиків на одній платформі.

Мета: розробити програмне забезпечення для підтримки та аналітики для продавців на торговельних майданчиках.

У розділі аналізу вимог до програмного забезпечення було проведено аналіз предметної області та наявних альтернатив розроблюваному продукту, сформульовано прецеденти використання компонентів системи, висунуто технічні вимоги до ПЗ.

У розділі моделювання та конструювання програмного забезпечення було здійснено опис архітектури системи, наведено інтерфейси їхньої взаємодії, проаналізовано рівень безпеки даних у запропонованому рішенні.

У розділі аналіз якості та тестування програмного забезпечення було описано сценарії тестування та вимоги до взаємодії структурних модулів.

У розділі впровадження та супроводу програмного забезпечення було описано алгоритм розгортання компонентів програмної системи.

КЛЮЧОВІ СЛОВА: ВЕБДОДАТОК, MYSQL, ІНТЕРФЕЙС, СЕРВЕРНА ЧАСТИНА, КЛІЄНТСЬКА ЧАСТИНА, ТОРГОВЕЛЬНИЙ МАЙДАНЧИК, МОДЕЛЬ С4.

ABSTRACT

The explanatory note of the diploma project consists of four sections, contains 20 tables, 18 figures and 15 sources – in total 56 pages.

The purpose of the diploma project is the creation software for the control of products from several trading platforms on one platform.

Objective: develop software for support and analytics for vendors on trading platforms.

Analysis of the subject area and available software alternatives was conducted in the software requirements analysis section. The use cases of the system components were formulated, technical requirements for the software were derived.

The software modelling and design section describes the architecture of the system components, interfaces of their interaction, and data security of the proposed solution.

The software quality analysis and testing section describes test cases and requirements for the interaction between the structural modules of the system.

In the software implementation and maintenance section, the algorithm for deploying software system components was described.

KEYWORDS: WEB APP, MYSQL, INTERFACE, SERVER SIDE, CLIENT SIDE, TRADING PLATFORM, C4 MODEL.

[illegible]

Факультет інформатики та обчислювальної техніки
Кафедра інформатики та програмної інженерії

“ЗАТВЕРДЖЕНО”

Завідувач кафедри

_____ Едуард ЖАРІКОВ

“ ____ ” _____ 2022 р.

**ПРОГРАМНЕ ЗАБЕЗПЕЧЕННЯ ДЛЯ ПІДТРИМКИ ТА АНАЛІТИКИ
ДЛЯ ПРОДАВЦІВ НА ТОРГОВЕЛЬНИХ МАЙДАНЧИКАХ**

Технічне завдання

КП.ІП-8233.045440.01.91

“ПОГОДЖЕНО”

Керівник проєкту:

_____ Олексій ФІНОГЕНОВ

Нормоконтроль:

_____ Катерина ЛІЩУК

Виконавець:

_____ Олександр ФІЛАТОВ

Київ – 2022

ЗМІСТ

1	НАЙМЕНУВАННЯ ТА ГАЛУЗЬ ЗАСТОСУВАННЯ	2
2	ПІДСТАВА ДЛЯ РОЗРОБКИ	3
3	ПРИЗНАЧЕННЯ РОЗРОБКИ	4
4	ВИМОГИ ДО ПРОГРАМНОГО ЗАБЕЗПЕЧЕННЯ	5
4.1	ФУНКЦІОНАЛЬНІ ВИМОГИ	5
4.2	НЕФУНКЦІОНАЛЬНІ ВИМОГИ	5
5	ВИМОГИ ДО ПРОГРАМНОЇ ДОКУМЕНТАЦІЇ	7
6	СТАДІЇ І ЕТАПИ РОЗРОБКИ	8
7	ПОРЯДОК КОНТРОЛЮ ТА ПРИЙМАННЯ	9

					КПІ.ІП-8233.045440.01.91	Арк.
Змін.	Арк.	№ докум.	Підп.	Дата.		1

1 НАЙМЕНУВАННЯ ТА ГАЛУЗЬ ЗАСТОСУВАННЯ

Назва розробки: Програмне забезпечення для підтримки та аналітики для продавців на торговельних майданчиках.

Галузь застосування:

Наведене технічне завдання поширюється на розробку програмного забезпечення «програмне забезпечення для підтримки та аналітики для продавців на торговельних майданчиках» [045440], котра використовується для взаємодії з торговельними майданчиками я OLX та AutoRia та призначена для ведення бізнесу та збору статистичних даних.

					КПІ.ІП-8233.045440.01.91	Арк.
Змін.	Арк.	№ докум.	Підп.	Дата.		2

2 ПІДСТАВА ДЛЯ РОЗРОБКИ

Підставою для розробки програмного забезпечення для підтримки та аналітики для продавців на торговельних майданчиках є завдання на дипломне проєктування, затверджене кафедрою інформатики та програмної інженерії Національного технічного університету України «Київський політехнічний інститут імені Ігоря Сікорського».

					КПІ.ІП-8233.045440.01.91	Арк.
Змін.	Арк.	№ докум.	Підп.	Дата.		3

3 ПРИЗНАЧЕННЯ РОЗРОБКИ

Розробка призначена для продавців які використовують один чи більше торговельних майданчиків.

Тобто продавець не повинен по черзі заходити в особисті кабінети на всі сайти торговельних майданчиків, щоб побачити результати торгівлі. Розроблене програмне забезпечення дозволяє використовуючи одну платформу бачити всі товари на вибраних торговельних майданчиках. Тобто інформацію про товари можна буде побачити на графіках статистик, які включають прибуток по всім каналам продаж та по кожному в окремість. Також є модивість проаналізувати статистику продажів, прослідкувати за оборотами, оцінити рентабельність. Тобто це прискорить процес аналітики за оголошеннями та поліпшить комфорт використання.

					КПІ.ІП-8233.045440.01.91	Арк.
Змін.	Арк.	№ докум.	Підп.	Дата.		4

4 ВИМОГИ ДО ПРОГРАМНОГО ЗАБЕЗПЕЧЕННЯ

4.1 Функціональні вимоги

- можливість реєстрації користувача;
- можливість авторизації користувача;
- можливість відновлення пароля користувача;
- можливість підключення торговельних майданчиків;
- можливість перегляду всіх товарів з підключених торговельних майданчиків;
- можливість фільтрувати товари;
- можливість додавати товар у список проданих товарів;
- можливість перегляду статистики за вибраними товарами;
- можливість перегляду загального прибутку та прибутку за останній місяць;
- можливість перегляду профілю користувача;
- можливість редагування профілю користувача.

4.2 Нефункціональні вимоги

- використання портів 80 HTTP, 443 HTTPS;
- кросплатформеність інтерфейсу;
- мінімальні параметри сервера для коректної роботи системи: CPU - 64bit x86, RAM - 4 GB;
- передача даних по мережі в зашифрованому вигляді;
- доступ до сервера по протоколу SSH за допомогою приватного ключа;
- відсутність прямого доступу до СУБД ззовні;
- автозапуск системи при перезавантаженні сервера.

Технічний стек являє собою (React.js, Node.js, MySQL). Тим самим вибрані технології роблять програмне забезпечення кросплатформеним, отже на сервері

					КПІ.ІП-8233.045440.01.91	Арк.
Змін.	Арк.	№ докум.	Підп.	Дата.		5

можна використовувати будь-яку з операційних систем (Linux, MacOS або Windows), але найкращим варіантом буде Linux, оскільки він є невимогливим до ресурсів, має продуману файлову систему та є дешевшим.

Логіка серверної частини ґрунтується на CRUD операціях, в яких складні структури даних відсутні, отже в якості API буде прагматичною ідеєю використати підхід REST.

Дані обов'язково мають передаватись в зашифрованому вигляді через мережу. Також доступ до СУБД ззовні повинен бути лише за відповідним секретним ключем. Щоб отримати доступ до сервера необхідно скористатися протоколом SSH.

Для розгортання проєкту необхідно буде вибрати сервіс для розгортання. Ним стане хмарний сервіс Amazon. Тому що Amazon - це класичний IaaS хостинг додатків. Він дозволяє вибрати мову програмування, платформу інтернет додатків, операційну систему також базу даних та інші необхідні послуги. Вибираючи AWS, отримуємо віртуальне середовище для завантаження програмного забезпечення та сервісів, потрібних для розробленого програмного забезпечення. Це спрощує процес міграції існуючих програм та зберігає можливість для створення нових рішень.

					КПІ.ІП-8233.045440.01.91	Арк.
Змін.	Арк.	№ докум.	Підп.	Дата.		6

5 ВИМОГИ ДО ПРОГРАМНОЇ ДОКУМЕНТАЦІЇ

Програмні модулі, котрі розробляються, повинні бути задокументовані, тобто тексти програм повинні містити всі необхідні коментарі.

Програмне забезпечення повинно мати супроводжувальну документацію.

До складу супроводжувальної документації повинні входити наступні документи:

- пояснювальна записка не менше ніж на 50 аркушах формату А4 (без додатків 5.3.2 – 5.3.6);
- технічне завдання;
- керівництво користувача;
- керівництво системного програміста;
- керівництво адміністратора;
- програма та методика тестування.

Графічна частина повинна бути виконана на 4 аркушах формату А3, котрі включаються у якості додатків до пояснювальної записки:

- схема структурна варіантів використання;
- схема структурна контексту програмного забезпечення;
- схема структурна контейнерів програмного забезпечення;
- схема структурна компонентів програмного забезпечення.

					КПІ.ІП-8233.045440.01.91	Арк.
Змін.	Арк.	№ докум.	Підп.	Дата.		7

6 СТАДІЇ І ЕТАПИ РОЗРОБКИ СТАДІЇ І ЕТАПИ РОЗРОБКИ

№	Назва етапу	Строк	Звітність
1.	Вивчення літератури за тематикою проєкту	10.03	
2.	Розробка технічного завдання	22.03	Технічне завдання
3.	Аналіз вимог та уточнення специфікацій	22.03	Специфікації програмного забезпечення
4.	Проектування структури програмного забезпечення, проектування компонентів	29.03	Схема структурна програмного забезпечення та специфікація компонентів (діаграма класів, схема алгоритму)
5.	Програмна реалізація програмного забезпечення	20.04	Тексти програмного забезпечення
6.	Тестування програмного забезпечення	14.05	Тести, результати тестування
7.	Розробка матеріалів текстової частини проєкту	25.05	Пояснювальна записка
8.	Розробка матеріалів графічної частини проєкту	20.05	Графічний матеріал проєкту
9.	Оформлення технічної документації проєкту	25.05	Технічна документація

7 ПОРЯДОК КОНТРОЛЮ ТА ПРИЙМАННЯ

Тестування розробленого програмного продукту виконується відповідно до «Програми та методики тестування».

**Пояснювальна записка
до дипломного проєкту**

на тему: Програмне забезпечення для підтримки та аналітики для продавців на
торговельних майданчиках

КПІ.ІП-8233.045440.02.81

ЗМІСТ

ПЕРЕЛІК УМОВНИХ ПОЗНАЧЕНЬ.....	3
ВСТУП.....	4
1 АНАЛІЗ ВИМОГ ДО ПРОГРАМНОГО ЗАБЕЗПЕЧЕННЯ.....	5
1.1. ОПИС І АНАЛІЗ ПРЕДМЕТНОЇ ОБЛАСТІ	5
1.2. ВИЗНАЧЕННЯ ПОКАЗНИКІВ АНАЛІТИКИ ЗА ОГОЛОШЕННЯМИ	6
1.3. РОЗРОБЛЕННЯ ФУНКЦІОНАЛЬНИХ ВИМОГ	6
1.4. РОЗРОБЛЕННЯ НЕФУНКЦІОНАЛЬНИХ ВИМОГ ТА ВИБІР ТЕХНОЛОГІЙ ДЛЯ	
1.5.1. Аналіз відомих програмних продуктів.....	20
1.6. ВИСНОВКИ ДО РОЗДІЛУ	26
2 МОДЕЛЮВАННЯ ТА КОНСТРУЮВАННЯ ПРОГРАМНОГО	
ЗАБЕЗПЕЧЕННЯ	27
2.1. МОДЕЛЮВАННЯ ПРОГРАМНОЇ СИСТЕМИ.....	27
2.2. АРХІТЕКТУРА ПРОГРАМНОГО ЗАБЕЗПЕЧЕННЯ	28
2.3. КОНСТРУЮВАННЯ ПРОГРАМНОГО ЗАБЕЗПЕЧЕННЯ.....	31
2.4. АНАЛІЗ БЕЗПЕКИ ДАНИХ	37
2.5. ВИСНОВКИ ДО РОЗДІЛУ	39
3 АНАЛІЗ ЯКОСТІ ТА ТЕСТУВАННЯ ПРОГРАМНОГО	
ЗАБЕЗПЕЧЕННЯ	40
3.1. АНАЛІЗ ЯКОСТІ ПЗ	40
3.2. ОПИС ПРОЦЕСІВ ТЕСТУВАННЯ	40
3.2.1. Компоненти що тестуються	40
3.2.2. Підхід до тестування.....	41
3.2.3. Критерій проходження тесту.....	41
3.3. ОПИС КОНТРОЛЬНОГО ПРИКЛАДУ	42
3.4. ВИСНОВКИ ДО РОЗДІЛУ	43

4	ВПРОВАДЖЕННЯ ТА СУПРОВІД ПРОГРАМНОГО ЗАБЕЗПЕЧЕННЯ	44
4.1.	РОЗГОРТАННЯ ПРОГРАМНОГО ЗАБЕЗПЕЧЕННЯ	44
4.2.	РОБОТА З ПРОГРАМНИМ ЗАБЕЗПЕЧЕННЯМ.....	45
	ВИСНОВКИ.....	55
	СПИСОК ВИКОРИСТАНИХ ДЖЕРЕЛ.....	56

ПЕРЕЛІК УМОВНИХ ПОЗНАЧЕНЬ

API	– Application programming interface, прикладний програмний Інтерфейс
C4	– Модель графічного запису для моделювання архітектури програмних систем
ОС	– Операційна система
БД	– База даних
UI	– Інтерфейс користувача

					КПІ.ІП-8233.045440.02.81	Арк.
Змін.	Арк.	№ докум.	Підп.	Дата.		3

ВСТУП

Щоб успішно торгувати на торговельних майданчиках, потрібно не лише зібрати дані про товари та прибутках, але й уміти їх аналізувати. Продавці, які вивчають графіки та знають, які позиції продаються краще, завжди на крок попереду конкурентів. Вони не витрачають ресурси на постачання нерентабельних товарів і економлять гроші.

Уважні продавці також прогнозують закупки та слідкують за залишками, щоб не допустити висловлення «не має на складі» на карточку товару та зберегти її місце в видачі. Без цього всього їх старання щодо розвитку продукції не принесе результату.

Торговельні майданчики або в сучасності вони мають назву «класифайди» – це ресурси з оголошеннями від фізичних та юридичних осіб з різними пропозиціями, що згруповані за темами. Серед найвідоміших українських «класифайдів» можна виділити: olx, izi.ua, ria.com, auto.ria.com, besplatka.ua та інші. Таким чином ці ресурси можна віднести до типу C2C.

При організації роботи продавця на подібних торговельних майданчиках проблемою є використання декількох систем одночасно, необхідна реєстрації на кожному з них, різна функціональність персональних кабінетів та неузгодження у форматах видачі статистики, що робить актуальним вирішення даної проблеми.

Метою роботи є розробка програмного забезпечення для підтримки та аналітики для продавців на торговельних майданчиках.

Задачі що повинні бути вирішені:

- провести аналіз існуючих аналогів;
- обрати архітектуру та визначити технології для реалізації програмного забезпечення;
- спроектувати базу даних;
- розробити функціональні та нефункціональні вимоги;
- провести тестування та оцінити якість програмного забезпечення.

1 АНАЛІЗ ВИМОГ ДО ПРОГРАМНОГО ЗАБЕЗПЕЧЕННЯ

1.1. Опис і аналіз предметної області

Програмне забезпечення підтримки та аналітики для продавців на торговельних майданчиках являє собою спеціалізований вебсайт, який належить торговій фірмі та призначений для прискорення та комфорту аналітики незалежно від торговельного майданчика.

Проблема яку вирішує дане програмне забезпечення – це аналіз оголошень одразу з декількох ресурсів.

Головні аспекти що були інтегровані до даного програмного забезпечення:

- глобальний збір статистичних даних;
- швидкий доступ одразу до декількох ресурсів;
- вибір торговельних майданчиків які підтримуються програмним забезпеченням;
- аналітика по кожному з оголошень;
- деактивування оголошення;
- додавання товару у список проданих.

Під'єднання торговельного майданчика на разі підтримує два вебресурси, а саме:

- OLX [1] – це так звана платформа онлайн оголошень, мета якої це об'єднати людей для купівлі або продажу чи обміну товарами та послугами. Сайт надає платформу для онлайн купівлі/продажу товарів та послуг для представників бізнесу та приватним особам. Користувачі які торгують на OLX платформі мають можливість розмістити свої товари, при попередній реєстрації за допомогою електронної пошти, мобільного телефону, або ж увійти через облікові записи у соціальних мереж. Також необхідно заповнити форму подачі оголошення, потрібно створити опис пропозиції, зробити фотографію та внести контактні дані. Ще є можливість відгукнутися на опублікований товар, для цього потрібно зв'язатися

					КПІ.ІП-8233.045440.02.81	Арк.
Змін.	Арк.	№ докум.	Підп.	Дата.		5

з його автором через онлайн-чат або за допомогою сайту чи номера телефону;

- AutoRia [2] – автомобільний портал - базар авто, з відкритим доступом корисної інформації та оголошеннями про продаж автомобілів в різних регіонах України.

Опишемо вхідні дані системи для розроблення реалізації повної чи часткової автоматизації цих процесів. Програмне забезпечення має можливість отримувати спеціальні токени для доступу до API “класифайдів”. Для доступу до Olx знадобиться лише підтвердити доступ до сервісу за допомогою системи oauth2. У випадку з AutoRia так не вийде через те, що даний ресурс не підтримує авторизацію через oauth2. Тобто знадобиться вручну надати секрети, а саме:

- api key;
- user Id.

Таким чином на виході буде отримано токен за яким можна буде отримати дані у форматі JSON з вибраних торговельних майданчиків.

1.2. Визначення показників аналітики за оголошеннями

Аналітика за оголошенням містить наступні показники:

- кількість переглядів;
- кількість спостережень;
- кількість взяттів номера телефону.

Також у кожного оголошення є свій строк дії, якщо він близький до кінця користувач отримує нагадування, для поновлення оголошення. Також продавець має можливість відстежити терміни дії на сторінці зі статистикою за товарами.

1.3. Розроблення функціональних вимог

До функціональних вимог можна віднести авторизацію. Під авторизацією мається на увазі реєстрація, вхід та вихід користувача. Також зроблено обробник помилок на валідність полів логін та пароль. Для того, щоб все коректно працювало і можна було правильно ідентифікувати того чи другого користувача

					КПІ.ІП-8233.045440.02.81	Арк.
Змін.	Арк.	№ докум.	Підп.	Дата.		6

використовується технологія JWT [3] токенів вона надала можливість кожному клієнту мати свій власний унікальний токен за яким серверна частина розуміє який саме користувач зараз у системі. Також у разі виникнення помилки користувач повинен бути виведений із системи.

Також виділимо під'єднання одного або усіх торговельних майданчиків, а саме OLX та AutoRia. У разі успішного під'єднання продавець відразу буде проінформований.

Продавець який під'єднав торговельний майданчик зможе побачити на головній сторінці список оголошень які він подав, їх можна буде відсортувати за тим чи іншим торговельним майданчиком або ж скористатися пошуком по товару. У кожній карточці товару має міститися найнеобхідніша інформація.

Що до статистики продавець зможе її побачити або ж на карточці товару або перейти на сторінку статистики та побачити актуальні графіки за такими критеріями як: кількість переглядів, кількість спостережень або ж кількість взяттів номера телефону. Також буде зроблений вивід найпопулярніших оголошень та оголошень у яких закінчується строк дії.

Користувач має можливість додати товар у список проданих. При підтвердженні вибору кількості проданих товарів інформація зберігається та не може бути виправлена, оскільки вона впливає на загальну статистику.

Розглянемо варіанти використання на діаграмі (додаток А) та у табличній формі (табл. 1.1 – 1.11):

Таблиця 1.1 – Варіант використання входу користувача в систему.

Назва	Вхід продавця в систему
Особливості	Продавець може проводити всі дії, які необхідні для входу, а саме: зареєструватися або авторизуватися.
Учасники	Продавець
Передумови	Продавець не авторизований

Таблиця 1.2 – Варіант використання авторизації користувача.

Назва	Авторизація продавця
Особливості	Продавець проходить авторизацію за допомогою вебдодатку.
Учасники	Не авторизований продавець.
Передумови	Продавець не має бути попередньо авторизованим
Постумови	Продавець проходить авторизацію в програмному забезпеченні
Основний сценарій	Продавець натискає кнопку позначену як «Увійти». Програмне забезпечення демонструє екран авторизації. Продавець заповнює власну інформацію для авторизації. А саме email та пароль. Система демонструє кнопку «Далі». Продавець натискає на кнопку «Далі». Система авторизує продавця.
Розширення сценаріїв	Програмне забезпечення виявляє, що дані внесені продавцем, не проходять валідацію даних за їх форматом, необхідних для авторизації, або продавець вже зареєстрований в системі за такими даними для авторизації. Програмне забезпечення демонструє продавцю повідомлення про відповідну помилку яка виводиться відповідним чином у правому кутку системи.

Таблиця 1.3 – Варіант використання реєстрації продавця в програмному забезпеченні.

Назва	Реєстрація продавця в програмному забезпеченні.
Особливості	Продавець може зберегти особисті дані які він вводить, а саме email та пароль, для подальшої авторизації у вебдодатку та його подальшому використанні.
Учасники	Неавторизований продавець
Передумови	Продавець не був попередньо зареєстрованим
Постумови	Продавець успішно пройшов реєстрацію
Основний сценарій	Програмне забезпечення демонструє екран для введення особистих даних для реєстрації продавця. Продавець вводить дані для подальшої авторизації та натискає кнопку “Далі”. Система надає продавцеві можливість підтвердити особисті дані. Тобто відбувається надсилання повідомлення на вказаний email.
Розширення сценаріїв	Програмне забезпечення виявляє наявність вже зареєстрованого акаунту за вказаними даними продавця, або ж виявляє помилку валідації даних.

Таблиця 1.4 – Варіант використання оновлення пароля продавця

Назва	Оновлення пароля продавця.
Особливості	Продавець може відновити свій пароль у разі його втрати.
Учасники	Авторизований продавець
Передумови	Авторизований продавець попередньо підтвердив та зареєстрував персональні дані для входу в програмне забезпечення.
Постумови	Продавець може змінити свій пароль у разі необхідності.
Основний сценарій	Продавець переходить на сторінку «Налаштування». Продавець натискає кнопку “Оновити пароль”. Програмне забезпечення виводить продавцеві форму для введення email. Продавець вводить email та натискає на кнопку “Відправити”. Програмне забезпечення дає продавцеві можливість змінити пароль, відіславши інструкцію на вказаний email.
Розширення сценаріїв	Користувач не вводить не вірні дані для проходження коректної процедури відновлення пароля. Програмне забезпечення може продемонструвати продавцеві повідомлення про відповідну помилку.

Таблиця 1.5 – Варіант використання підключення торговельних майданчиків.

Назва	Підключення торговельних майданчиків
Особливості	Продавець має можливість приєднати один або більше торговельних майданчиків.
Учасники	Продавець
Передумови	Продавець повинен бути авторизованим. Повинні бути попередньо створені облікові записи на тих торговельних майданчиках до яких продавець бажає під'єднатися.
Постумови	Продавець успішно під'єднав вибрані торговельні майданчики.
Основний сценарій	Програмне забезпечення демонструє користувачеві успішне повідомлення про підключення вибраного торговельного майданчика.
Розширення сценаріїв	Створений обліковий запис на вибраному торговельному майданчику видається під час спроби підключення. Програмне забезпечення демонструє користувачеві помилку у разі виникнення підозрілих дій або при виникненні відповідної помилки на серверній частині програмного забезпечення.

Таблиця 1.6 – Варіант використання отримання доступних товарів

Назва	Отримання доступних товарів
Особливості	Продавець має можливість переглядати список доступних продуктів.
Учасники	Продавець
Передумови	Продавець повинен бути авторизованим. Вивід товарів відбувається тільки коли продавець має принаймні один товар на одному з торговельних майданчиків OLX або AutoRia.
Постумови	Продавець отримав список доступних продуктів.
Основний сценарій	Програмне забезпечення демонструє продавцеві список продуктів з усіх підключених торговельних майданчиків.
Розширення сценаріїв	Продукт було видалено у проміжку часу між успішною авторизацією користувача в одній із систем торговельних майданчиків та переходом на сторінку зі списком продуктів. Програмне забезпечення демонструє продавцю помилку у разі виникнення підозрілих дій або при виникненні відповідної помилки на серверній частині програмного забезпечення.

Таблиця 1.7 – Варіант використання – додавання товару у список проданих товарів.

Назва	Додавання товару у список проданих товарів.
Особливості	Продавець має можливість додати вибрану кількість продуктів у список проданих.
Учасники	Продавець
Передумови	Продавець повинен бути авторизованим у системі та мати у продажу хоча б один товар на одному з торговельних майданчиків.
Постумови	Продавець додав вибрану кількість продуктів у список проданих.
Основний сценарій	Продавець вибирає продукт. Продавець натискає на кнопку «Продати». Відображається модальне вікно у якому продавець задає кількість проданих товарів та натискає на кнопку «Далі».
Розширення сценаріїв	Продукт може бути видалено у проміжку часу між успішною авторизацією продавця в одній із систем торговельних майданчиків та переходом на сторінку зі списком продуктів. Система може продемонструвати продавцеві відповідну помилку на не проходження валідації.

Таблиця 1.8 – Варіант використання – деактивування продукту.

Назва	Деактивування продукту
Особливості	Продавець має можливість деактивувати вибраний товар зі списку.
Учасники	Користувач
Передумови	Продавець повинен бути авторизованим у системі та мати у продажу хоча б один товар на одному з торговельних майданчиків OLX або AutoRia.
Постумови	Продавець додав вибрану кількість продуктів у список проданих.
Основний сценарій	Продавець вибирає продукт. Користувач натискає на кнопку «Deactivate». Відображається модальне вікно у якому користувач повинен підтвердити свою дію.
Розширення сценаріїв	Продукт міг бути видаленим у проміжку часу між успішною авторизацією продавця в одній із систем торговельних майданчиків та переходом на сторінку зі списком продуктів. Програмне забезпечення має можливість продемонструвати продавцеві відповідні помилки на не проходження валідації даних у програмному забезпеченні.

Таблиця 1.9 – Варіант використання – перегляд статистики.

Назва	Отримання доступних продуктів
Особливості	Продавець має можливість переглядати статистику проданих та активних товарів.
Учасники	Продавець
Передумови	Продавець повинен бути авторизованим у системі та мати у продажу хоча б один товар на одному з торговельних майданчиків OLX або AutoRia.
Постумови	Продавець отримав статистику по товарах.
Основний сценарій	Програмне забезпечення демонструє продавцеві статистику продуктів з усіх підключених торговельних майданчиків та проданих товарів з сервісів OLX та AutoRia.
Розширення сценаріїв	Товар було видалено у проміжку часу між успішною авторизацією продавця в одній із систем торговельних майданчиків та переходом на сторінку зі списком продуктів. Програмне забезпечення може продемонструвати продавцеві відповідну помилку на не проходження валідації даних у системі.

Таблиця 1.10 – Варіант використання – перегляд профілю користувача.

Назва	Отримання доступних продуктів
Особливості	Продавець має можливість переглядати дані свого профілю.
Учасники	Продавець
Передумови	Продавець авторизований.
Постумови	Продавець отримав дані профілю.
Основний сценарій	Система демонструє користувачеві дані профілю.
Розширення сценаріїв	Система демонструє оповіщення про відповідну помилку.

Таблиця 1.11 – Варіант використання – фільтрація товарів.

Назва	Фільтрація товарів
Особливості	Продавець має можливість фільтрувати товари за вибраним сервісом.
Учасники	Продавець
Передумови	Продавець повинен бути авторизованим.
Постумови	Продавець відфільтрував товари за вибраним сервісом.
Основний сценарій	Продавець натискає на кнопку «Filter» на сторінці списку всіх продуктів та обирає торговельний.
Розширення сценаріїв	Система демонструє продавцеві повідомлення про відповідну помилку.

1.4. Розроблення нефункціональних вимог та вибір технологій для розробки

Нефункціональні вимоги для продавця у розробленому програмному забезпеченні є досить простими. Тобто для комфортного користування програмним забезпеченням користувачеві потрібен звичайний пристрій який має вихід в інтернет. В такому випадку можна розробити SaaS вебдодаток з використанням бібліотеки React.js [4]. Таким чином програмне забезпечення отримує наступні переваги:

- кросплатформність;
- добре працює з SEO;
- продуктивність;
- зручна підтримка.

Також React.js використовує компонентно-орієнтований підхід який дає можливість з легкістю редагувати існуючі компоненти та здатність перевикористати код, тим самим це перетворює React розробку в процес покращення написання програмного забезпечення. Ті компоненти, які були створені у роботі над програмним забезпеченням, не мають додаткових залежностей. Тобто, ніщо не заважає їх перевикористати.

Бізнес-логіка розміщена на серверній стороні програмного забезпечення. Для цього можна використати Node.js [5]. Ця платформа була обрана як основна, тому що вона є наразі найпопулярнішою для написання серверної частини програмного забезпечення, використовує мову програмування Javascript [6]. Її перевагами є:

- швидкість та продуктивність – в його не блокуючі операції введення-виведення роблять середовище одним із найшвидших доступних варіантів. Код працює досить швидко та значно покращує все середовище виконання, а це своєю чергою пов'язано з його роздільною системою. Також він працює на движку JavaScript V8 який розроблений Google. Серверна частина, яка написана за

допомогою Node.js має сумісність plug and play що сприяє швидкості та продуктивності розроблюючого додатку;

- ефективне кешування – це одна з ключових переваг Node.js. Оскільки, він має досить потужну здатність кешувати дані. В той час коли до додатка надходять запити, вони мають здатність кешуватися у пам'яті програми;
- гнучкість – гнучкість програмування це одне з найбільших переваг Node.js. Оскільки коли вносять зміни в Node.js додаток, це може вплинути лише на один вузол. І це чудово не тільки для початкової збірки, але й для подальшого обслуговування. Також це стосується того, коли поєднується JSON з Node.js, таким чином досить легко обмінюватися даними між клієнтським сервером та вебсервером. Також можна використовувати API для вбудовування TCP, HTTP, DNS тощо;
- здатність до масштабування проєкту;
- величезне ком'юніті розробників.

Node.js також дозволяє підключати інші модулі, написані іншими людьми на різних мовах програмування. Для цієї задачі створені різні пакетні менеджери, але в даній роботі було використано npm package manager. Оскільки він дає змогу комфортно керувати кількома версіями коду та його залежностями.

Сервер створено на основі Express.js [7], через те, що він має велику площу і являє собою дуже тонкий шар абстракції над даним вузлом для створення додатків на серверній стороні програмного забезпечення. Він повністю підключається і має величезну спільноту. Також він побудований з використанням передової та прогресивної технології (ES6), за допомогою якої з'являються очевидні переваги перед старими фреймворками.

У даного програмного забезпечення немає жорстких вимог що до масштабованості бази даних оскільки всі дані зберігаються виключно в базах торговельних майданчиків, а зв'язок відбувається через їхнє API. В такому випадку було вирішено вибрати реляційну СУБД MySQL [8].

					КПІ.ІП-8233.045440.02.81	Арк.
Змін.	Арк.	№ докум.	Підп.	Дата.		18

Головною метою було забезпечити цілісність інформації в базі даних. Потрібно приділити увагу персональним даним користувачів. Для конфіденційних даних користувача використовується кілька кроків шифрування.

- по-перше, ключ шифрування даних шифрується за допомогою AWS Encryption SDK використовуючи AES у режимі GCM з 256 бітами розмір ключа;
- по-друге, ключ, який використовувався для шифрування ключа шифрування даних, зберігається в AWS KMS;
- по-третє, зашифрований ключ і дані кодуються в base64 і зберігаються в базі даних.

Тобто таким чином вдалося запобігти порушенню даних і витоків даних, оскільки, коли зломисник скомпрометує дані одного користувача, неможливо отримати доступ до даних інших користувачів, використовуючи скомпрометовані, ключ шифрування, що зберігається в AWS KMS, і ключ шифрування даних який захищає дані користувача.

Мінімальні технічні параметри сервера:

- CPU – 64bit x86;
- RAM – 4 GB.

Рекомендовані параметри сервера:

- CPU – Multi Core 64bit x86;
- RAM – 8 GB or higher.

Технічний стек являє собою (React.js, Node.js, MySQL). Тим самим вибрані технології роблять програмне забезпечення кросплатформеним, отже на сервері можна використовувати будь-яку з операційних систем (Linux, MacOS або Windows), але найкращим варіантом буде Linux, оскільки він є невимогливим до ресурсів, має продуману файлову систему та є дешевшим.

Логіка серверної частини ґрунтується на CRUD операціях, в яких складні структури даних відсутні, отже в якості API буде прагматичною ідеєю використати підхід REST.

					КПІ.ІП-8233.045440.02.81	Арк.
Змін.	Арк.	№ докум.	Підп.	Дата.		19

Дані обов'язково мають передаватись в зашифрованому вигляді через мережу. Також доступ до СУБД ззовні повинен бути лише за відповідним секретним ключем. Щоб отримати доступ до сервера необхідно скористатися протоколом SSH.

Для розгортання проєкту необхідно буде вибрати сервіс для розгортання. Ним стане хмарний сервіс Amazon [9]. Тому що Amazon - це класичний IaaS хостинг додатків. Він дозволяє вибрати мову програмування, платформу інтернет додатків, операційну систему також базу даних та інші необхідні послуги. Вибираючи AWS, отримуємо віртуальне середовище для завантаження програмного забезпечення та сервісів, потрібних для розробленого програмного забезпечення. Це спрощує процес міграції існуючих програм та зберігає можливість для створення нових рішень.

1.5. Аналіз альтернатив

Профільтрувавши ринок було знайдено схожі аналоги такі як merchantspring.io та moneyplace.io, але вони беруть інформацію з маркетплейсів, а це вже інший тип продажів так званий B2C – “business to consumer” це операція, яка існує між бізнесом і кінцевим споживачем. Вона може включати будь-який процес продажу, коли продаж товарів і надання послуг компанією здійснюється безпосередньо кінцевому користувачеві.

1.5.1. Аналіз відомих програмних продуктів

Найбільш популярні платформи для збору статистичних даних:

Moneyplace

Moneyplace шукає популярні та маржинальні товари на Ozon, Aliexpress, Wildberries, СберМегаМаркет, Яндекс.Маркет та KazanExpress, інтерфейс зображено на рисунку 1.1.

					КПІ.ІП-8233.045440.02.81	Арк.
Змін.	Арк.	№ докум.	Підп.	Дата.		20

ЗБРОГ ЭДЖЭНСИ ООО

Главная / Продавцы / **Продавец**

Выручка

9,733,297

Разница со вчерашним днем:

84% ↑

Продаж

12,143

Разница со вчерашним днем:

133% ↑

Состояние склада

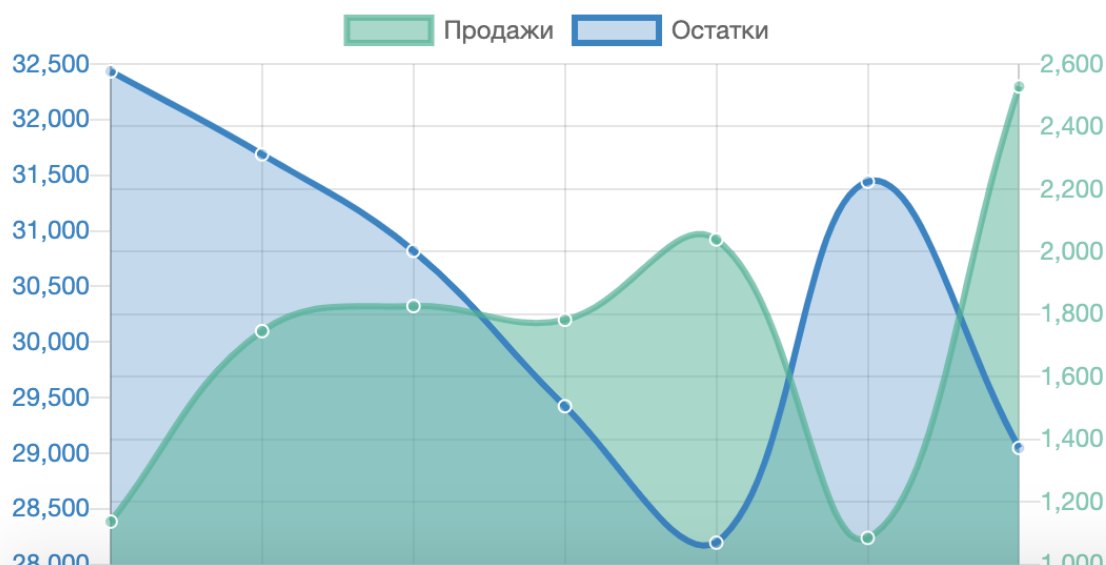


Рисунок 1.1 – Интерфейс Moneyplace

Сервіс шукає новинки та тренди, той товар, який точно не залежить. Він зручний не тільки при пошуку ніші, але і при аналізі магазину, що вже діє.

Підключення магазину API дозволяє аналізувати продажі продавця. Видно статистику виторгу, за середнім чеком, залишками. А також сервіс контролює та розраховує поставки, рекомендовані дати надходять повідомленнями. Тарифи на використання різні: від 1000 гривень до 30000 гривень на місяць, пакети відрізняються різною кількістю користувачів, періодами аналізу, кількістю вивантажених рядків, наявністю послуг персонального аналітика.

Змін.	Арк.	№ докум.	Підп.	Дата.

КПІ.ІП-8233.045440.02.81

Арк.

21

Недоліком даного продукту є тим що даний сервіс не спеціалізується на українському ринку.

Маяк

Сервіс Маяк створений для пошуку топових товарів на Wildberries через аналіз кількості продажів інших продавців, їх втрачених продажів, статистики з виторгу та продажу, динаміки отриманих товарів у категорії та в пошуку за запитам, інтерфейс зображено на рисунку 1.2.

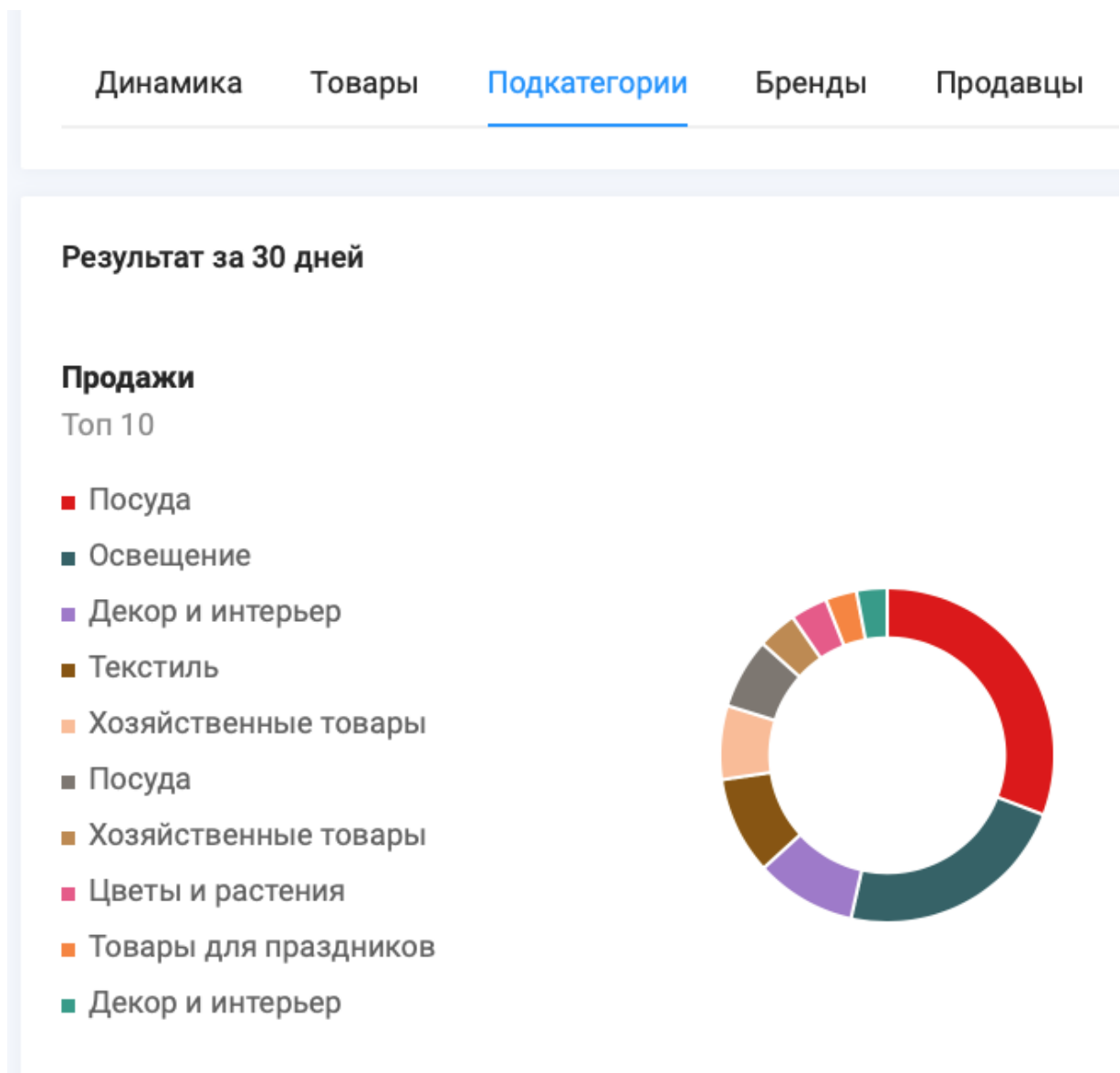


Рисунок 1.2 – Интерфейс Маяк

Зручність сервісу у розширенні для браузера Chrome. З ним основні дані статистики проглядаються у картці товару на Wildberries. А ще в сервісі є бот в Telegram, який у відповідь на відправлене посилання на картку товару, надсилає графік з даними про виторг, замовлення за останні 30 днів та залишки.

Безплатний тариф із невеликим функціоналом – незамінний для абсолютних новачків з обмеженим бюджетом для визначення ніші. За 1000 гривень на місяць доступний стандартний тариф. Для просунутих продавців є тариф "Професійний". Однією з ключових фішок даного тарифу є вибір ключових слів для описів.

Недоліком даного продукту є тим що даний сервіс не спеціалізується на українському ринку. Також цей продукт підтримує тільки один торговельний майданчик, а це є найменшим показником серед конкурентів.

Shopstat

Shopstat – це безплатний сервіс аналітики збирає дані для аналізу з Wildberries та Ozon. Яндекс.Маркет та AliExpress у розробці. Показує аналітичні дані щодо будь-яких товарів, не важливо, ваші вони чи іншого продавця, інтерфейс зображено на рисунку 1.3.

					КПІ.ІП-8233.045440.02.81	Арк.
Змін.	Арк.	№ докум.	Підп.	Дата.		23

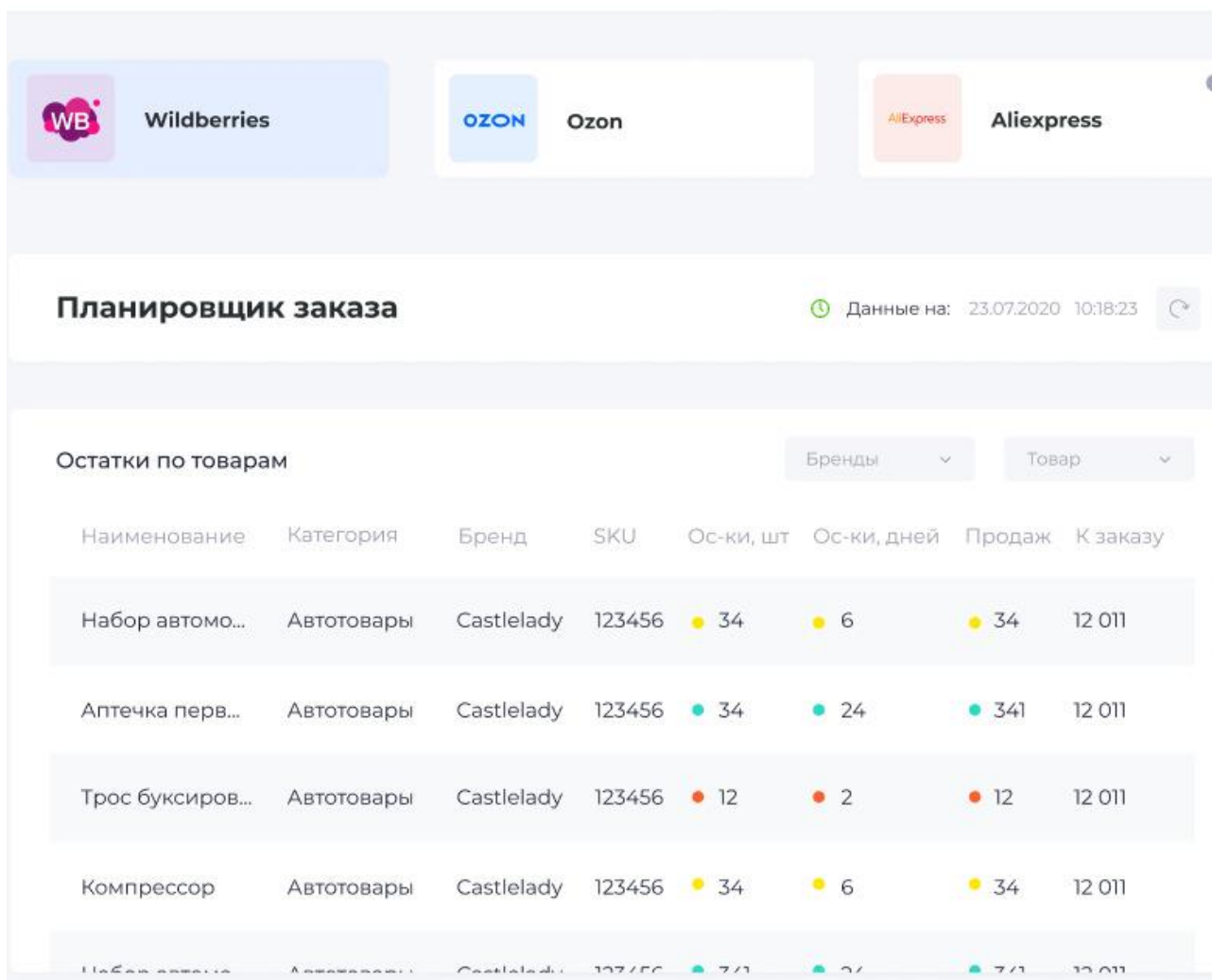


Рисунок 1.3 – Интерфейс Shopstat

У тому числі можна відстежити зміни цін, вплив залишку товару на прибуток, топ товарів за період. Повна статистика категорії фільтрується за рейтингом, виторгом, кількістю, ціною, залишком. З сервісом дізнаєтеся попит без тестового постачання товару, перевірте торгові гіпотези та нарешті оберете нішу.

Дані географії продажів за складами показуються з урахуванням розмірів і кольорів: легко розрахувати оборотність і налагодити постачання регіони, де товар користується популярністю.

Інформація про продаж оновлюється щомиті, тому будь-який запит статистики принесе лише актуальні дані.

Недоліком даного продукту є тим що даний сервіс не спеціалізується на українському ринку.

Merchantspring

Merchantspring – це єдина платформа для керування продуктивністю. В яку включаються Amazon, Walmart, Shopify та понад 75 ринків у всьому світі, інтерфейс зображено на рисунку 1.4.

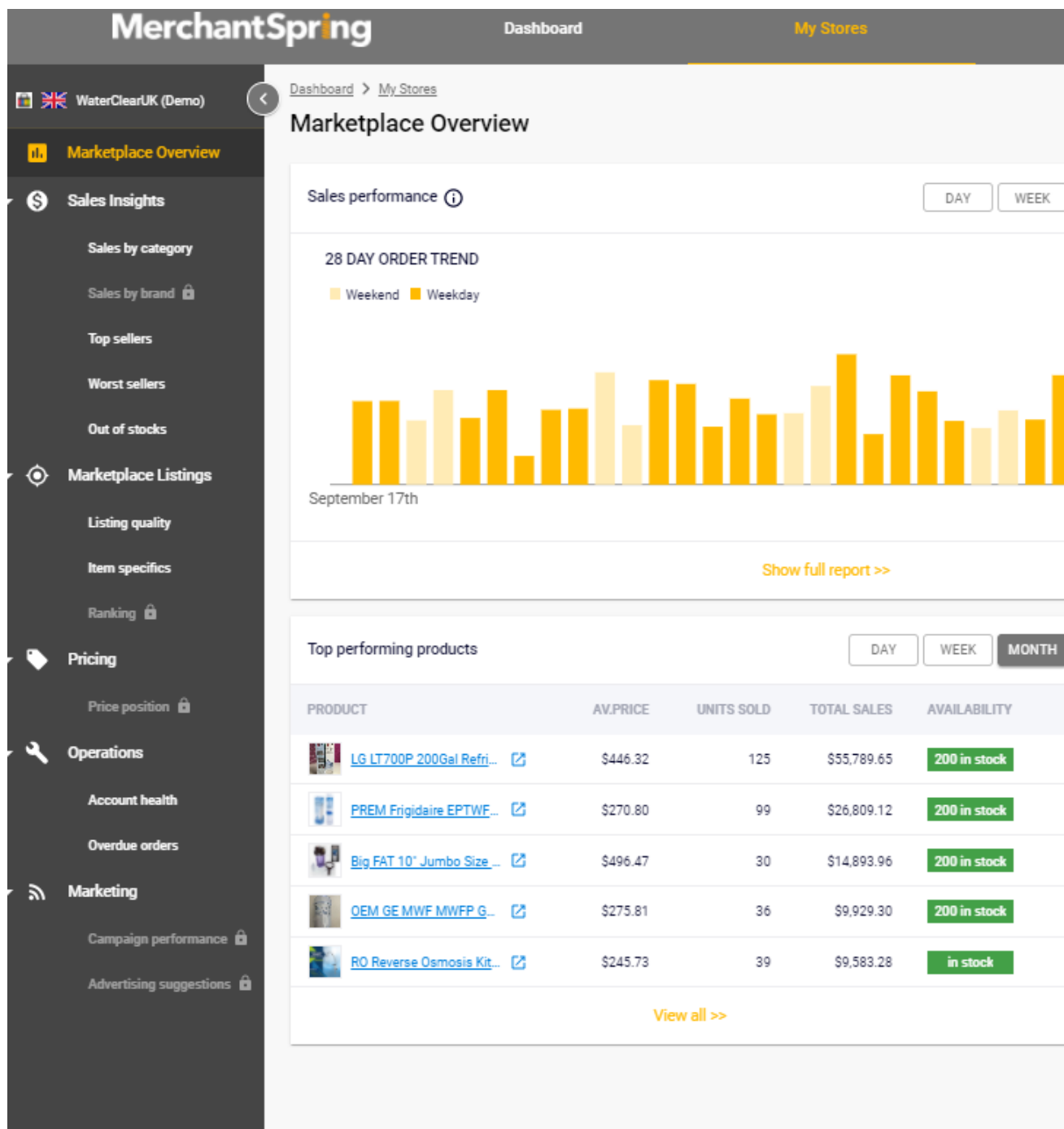


Рисунок 1.4 – Інтерфейс Merchantspring

Платформа дає можливість відстежувати все, де б ви не були. Можна стежити за тенденціями продажів, якістю списку та коефіцієнтами конверсії трафіку, коли ви в дорозі, за допомогою зручного мобільного центру та центру сповіщень.

Всі перелічені платформи безпосередньо конкурують між собою, але для розробленого програмного забезпечення вони є дуже близькими, але не є конкурентами через те, що вони не орієнтуються на український ринок та не мають тип продажу C2C.

Недоліком даного продукту є тим що даний сервіс не спеціалізується на українському ринку, та має досить складну архітектуру для користувача.

Веб застосунок підтримує усі сучасні браузері. Бізнес-логіка в переважній більшості розташована на серверній частині програмного забезпечення.

1.6. Висновки до розділу

Проведено аналіз до вимог програмного забезпечення. Таким чином серед розглянутих альтернатив не було знайдено тих що задовольняють висунуті функціональні та нефункціональні вимоги оскільки вони не забезпечують вихід на український ринок. Тим самим розроблене програмне забезпечення робить його актуальним через те, що воно орієнтується на український ринок та має тип торговельних майданчиків C2C. В розділі було визначено перелік необхідних програмних засобів для реалізації проєкту. На основі аналізу було вибрано клієнт-серверну архітектуру, де для забезпечення серверної логіки вибрано Node.js, для клієнтської React.js у якості СУБД обрано MySQL.

2 МОДЕЛЮВАННЯ ТА КОНСТРУЮВАННЯ ПРОГРАМНОГО ЗАБЕЗПЕЧЕННЯ

2.1. Моделювання програмної системи

Для моделювання програмної системи було вирішено використати модель С4 [10]. Оскільки вона є простим методом графічного запису для моделювання архітектури програмних систем. Модель С4 заснована на структурній декомпозиції системи на контейнери та компоненти які спирається на методи що існують для моделювання, такі як Unified Modeling Language (UML) або Entity Relation Diagrams (ERD).

Точки огляду організовані відповідно до їх ієрархічного рівня:

- діаграми контексту (рівень 1): вони показують систему у масштабі та її взаємовідносини з користувачами та іншими системами;
- діаграми контейнерів (рівень 2): вони розбивають систему на взаємопов'язані контейнери. Контейнер — це підсистема, що виконується і розгортається;
- діаграми компонентів (рівень 3): вони поділяють контейнери на взаємопов'язані компоненти та зв'язують компоненти з іншими контейнерами або іншими системами;
- діаграми коду (рівень 4): вони надають додаткові відомості про дизайн архітектурних елементів, які можна порівняти з кодом.

Було створено діаграми для перших трьох рівнів.

Діаграма контексту системи (додаток Б) зображає продавця, основну систему та торговельні майданчики з якими взаємодіє розроблене програмне забезпечення.

Торговельні майданчики є сторонніми сервісами тому вони були винесені за межі програмного забезпечення.

Діаграма контейнерів програмного забезпечення (додаток В). На цьому ієрархічному рівні можна побачити з якими контейнерами працює розроблене програмне забезпечення. Тобто вебінтерфейс комунікує тільки з серверною

частиною, а серверна частина своєю чергою має зв'язок з базою даних та сторонніми сервісами такими як OLX та AutoRia. Цей зв'язок зміг бути можливим завдяки безплатному API яке надає кожна з систем.

Діаграма компонентів програмного забезпечення (додаток Д). На цьому ієрархічному рівні можна побачити, що вся бізнес-логіка програмного забезпечення знаходиться на серверній стороні програмного забезпечення. Тобто показано як авторизація та відновлення пароля комунікують з “security” компонентом який своєю чергою шифрує дані у три етапи. Перший етап це приведення до однакової довжини за допомогою алгоритму SHA-512 [11]. Другий етап це шифрування даних за допомогою argon2. Та останнім кроком своєю чергою став алгоритм AES-256 [12] за допомогою ключа який зберігався на AWS KMS – (Amazon Key Management Service). Також демонструється за яким принципом торговельні майданчики комунікують з розробленим програмним забезпеченням. Тобто щоб отримати будь-яку інформацію про продавця на тому чи іншому торговельному майданчику потрібна відповідна авторизація. При успішній авторизації access токени будуть зберігатися в localStorage. Але через те, що дія цього токена відносно мала (близько 20 хв) потрібно його постійно оновлювати. Тож було прийняти рішення зберігати токен для оновлення у базі даних оскільки він має значно більший термін дії (близько 1,5 року) відповідно будить досить довго не змінний.

2.2. Архітектура програмного забезпечення

На рисунку 2.1 зображена схема клієнт-серверної архітектури системи. На схемі демонструється як взаємодіє кожен з додатків та як фігурує бізнес-логіка у кожному з процесів.

					КПІ.ІП-8233.045440.02.81	Арк.
Змін.	Арк.	№ докум.	Підп.	Дата.		28

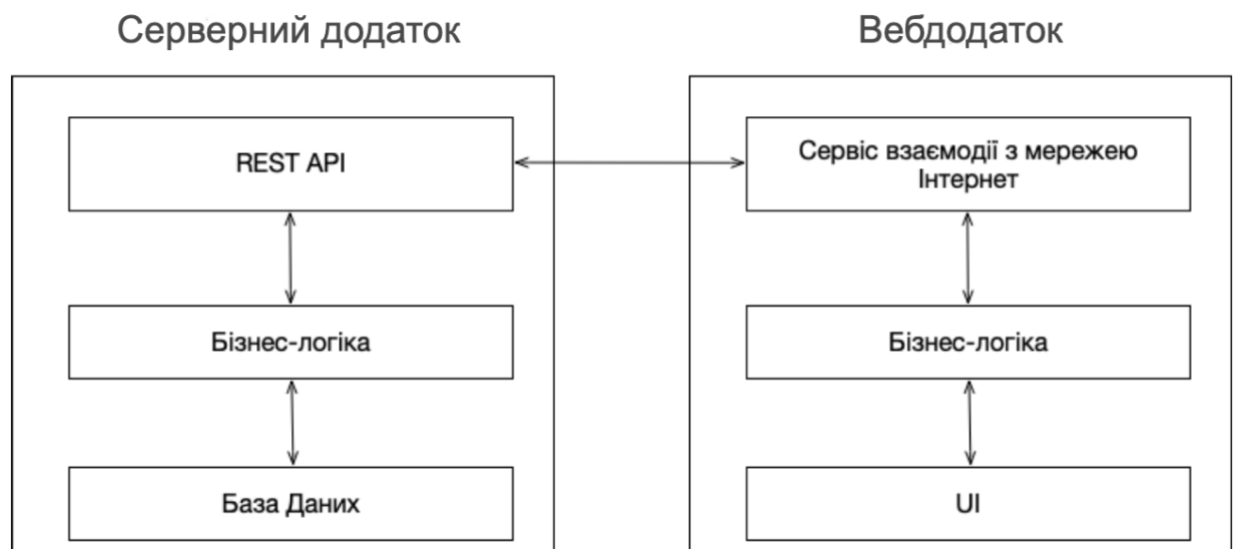


Рисунок 2.1 – Схема клієнт-серверної архітектури системи

Взаємодія між компонентами системи здійснюється за допомогою HTTP запитів до REST API, які підтримуються сервером. REST (Representational State Transfer) – це спосіб створення API за допомогою протоколу HTTP.

Технологію REST API застосовують скрізь, де користувачеві сайту або вебпрограмі потрібно надати дані з сервера. Наприклад, при початковому натисканні іконки з відео на youtube, REST API проводить операції та запускає ролик із сервера у браузері. Зараз це найпоширеніший спосіб організації API. Він витіснив раніше популярні методи SOAP і WSDL.

При реалізації буде виконано 7 основних принципів роботи з REST API:

- відділення клієнта від сервера (Client Server). Клієнт – це безпосередній інтерфейс вебсайту або ж програми. У REST API код запитів залишається на стороні покупця, а код доступу до інформації – на стороні серверної частини. Це спрощує керування API та дозволяє просто виносити інтерфейс користувача на іншу платформу і виділяє ймовірність чимось іншого масштабувати серверне заощадження даних;
- відсутність запису стану клієнта (Stateless). Сервер не повинен зберігати інформацію про стан (проведені операції) клієнта. Кожен запит від вебдодатку має містити тільки інформацію, яка була б потрібна для отримання даних від серверної частини;

- кешування (Cachable). У даних запиту має бути зазначено, чи потрібно кешувати дані або ж іншими словами зберігати у спеціальному буфері для частих запитів. Якщо така вказівка є, клієнт отримає право звертатися до цього буфера за потреби;
- єдність інтерфейсу (Uniform Interface). Усі дані мають проходити через 1 URL адресу стандартними за протоколами, наприклад, HTTP. Це полегшує архітектуру сайту або програми й робить взаємодію з сервером більш зрозумілою;
- багаторівневність системи (Layered System). У RESTful сервері можуть розташовуватися різних рівнях, у своїй кожен сервер взаємодіє лише з найближчими рівнями та пов'язаний запитами коїться з іншими;
- передача коду на запит (Code on Demand). Сервери можуть надсилати покупцеві код. Наприклад артільний код програми або вебсайт стає складніше тільки при необхідності;
- початок від нульової позначки (Starting with the Null Style). Замовник розуміє лише 1 точку входу на сервер. Наступні можливості взаємодії забезпечуються сервером. REST API базується на протоколі передачі гіпертексту HTTP (Hypertext Transfer Protocol). Це стандартний протокол в інтернеті, створений для передачі гіпертексту. Також за допомогою HTTP можна надсилати будь-які інші типи даних.

У розробленому REST API є 4 HTTP методи, за допомогою яких клієнт відправляє на сервер відповідний тип запитів:

- GET – отримання інформації про дані або список об'єктів;
- POST – додавання або зміна даних;
- PUT – оновлення даних;
- DELETE – видалення даних;

Такі запити ще можна назвати ідентифікаторами CRUD, які контролюють основну частину операцій між клієнтом та серверною частиною програмного забезпечення, приклад продемонстровано на рисунку 2.2.

					КП.ІП-8233.045440.02.81	Арк.
Змін.	Арк.	№ докум.	Підп.	Дата.		30

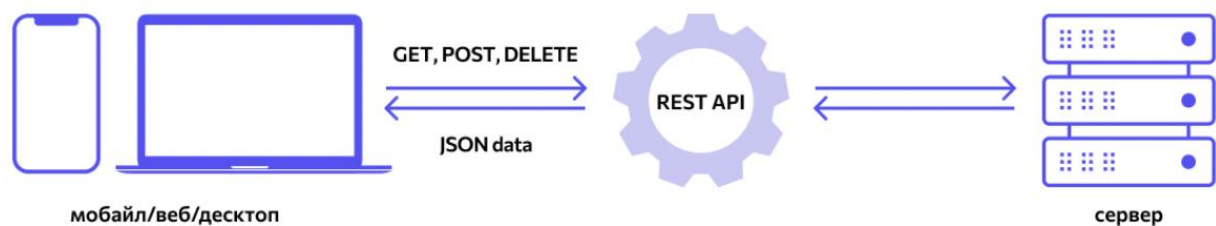


Рисунок 2.2 – Архітектура REST API

На даному рисунку можна чітко побачити як клієнт комунікує з сервером за допомогою REST API. Важливо зазначити що сама архітектура REST не прив'язана до конкретних технологій і протоколів, але в реаліях сучасного Інтернету, побудова RESTful API майже завжди має на увазі використання HTTP і будь-яких поширених форматів представлення ресурсів, наприклад JSON, або, менш популярного сьогодні, XML.

2.3. Конструювання програмного забезпечення

Для взаємодії з клієнтським застосунком використовують наступні інтерфейси:

- authentication – інтерфейс для процесів авторизації, реєстрації, відновлення та зміни пароля;
- statistics – інтерфейс для отримання статистики продуктів;
- products – інтерфейс для підключення торговельних майданчиків та виводу продуктів;
- account – інтерфейс для отримання даних профілю.

За допомогою інтерфейсу можна вказати, що саме повинен виконувати клас, що його реалізує, але не як це робити. Спосіб реалізації вибирає сам клас. Інтерфейси не здатні зберігати дані стану. Інтерфейси - це один із механізмів реалізації принципу поліморфізму "один інтерфейс, кілька методів"

Таблиця 2.1 – Методи інтерфейсу Authentication.

Назва метода	Аргументи	Значення, яке повертається	Призначення методу	Коди помилок
registration	email, password	HTTP Code	Зареєструвати користувача у системі	001 002
login	email, password	access token	Увійти у застосунок із вказаними email та паролем	001
reset_password	email		Оновити пароль за вказаним email	003

Таблиця 2.2 – Коди помилок інтерфейсу Authentication.

Код помилки	Системна назва помилки	Значення помилки
001	INVALID_CREDENTIALS	Помилка в авторизації. Не правильний email або пароль.
002	EMAIL_ALREADY_IN_USE	Email вже використовується для іншого користувача.
003	INVALID_EMAIL	Користувача з таким email не існує.
004	NOT_AUTHENTICATED	Користувач не аутентифікований.

Таблиця 2.3 – Методи інтерфейсу Statistics.

Назва метода	Аргументи	Значення, яке повертається	Призначення методу	Коди помилок
GetActiveStatistics	productid (optional), olx_access_token та/або ria_access_token	Масив статистики	Отримати дані статистики з торговельних майданчиків	101 102 103
GetSoldStatistics	productid (optional)	Масив статистики	Отримати дані статистики проданих товарів.	101

Таблиця 2.4 – Коди помилок інтерфейсу Statistics.

Код помилки	Системна назва помилки	Значення помилки
101	PRODUCTS_NOT_EXIST	Повинен бути хоча б один продукт для виведення статистики.
102	RIA_TOKEN_NOT_EXIST	Невірний токен при намаганні стягування даних з торговельного майданчика.
103	OLX_TOKEN_NOT_EXIST	Невірний токен при намаганні стягування даних з торговельного майданчика.

Таблиця 2.5 – Методи інтерфейсу Products.

Назва метода	Аргументи	Значення, яке повертається	Призначення методу	Коди помилок
CreateSession	riaUserId, riaToken або olx_auth_code	HTTP Code	Авторизація з сервісами OLX та/або AutoRIA за допомогою систем API.	201 202
GetProducts	olx_access_token та/або ria_access_token	Масив усіх продуктів незалежно від торговельного майданчика	Отримання усіх товарів з попередньо підключених торговельних майданчиків.	201 202
StopProduct	olx_access_token та/або ria_access_token, productId	HTTP Code	Деактивувати товар на тому чи іншому торговельному майданчику	203
SoldProduct	olx_access_token та/або ria_access_token, productid, count	HTTP Code	Додати продукт у список проданих товарів. Ця діє не може бути повернена до попереднього значення.	203 204

Таблиця 2.6 – Коды помилок інтерфейсу Authentication

Код помилки	Системна назва помилки	Значення помилки
201	INVALID_OLX_TOKEN	Помилка в авторизації до торговельного майданчика OLX. Не правильний access_token.
202	INVALID_RIA_TOKEN	Помилка в авторизації до торговельного майданчика AutoRia. Не правильний access_token.
203	INVALID_PRODUCT	Продукту з таким ідентифікатором не існує
204	OUT_OF_STOCK	Даного продукту не має в наявності.

Таблиця 2.7 – Методи інтерфейсу Account.

Назва метода	Аргументи	Значення, яке повертається	Призначення методу	Коди помилок
GetProfile	token	Об'єкт даного юзера	Переглянути дані профілю поточного користувача	301
UpdateProfile	token	HTTP Code	Оновити дані профілю такі як фото або пароль	301 302 303

Таблиця 2.8 – Коды помилок інтерфейсу Account.

Код помилки	Системна назва помилки	Значення помилки
301	USER_NOT_EXIST	Користувача не знайдено не вірний токен
302	IMG_WRONG_FORMAT	Не вірний формат картинки
303	IMG_SIZE_LIMIT	Ліміт розміру картинки перевищено

Спроектована база даних забезпечує зберігання інформації для користувачів системи, інформацію о реєстрації на торговельних майданчиках, фіксує факт продажу товару для оновлення пропозицій (рис. 2.3).

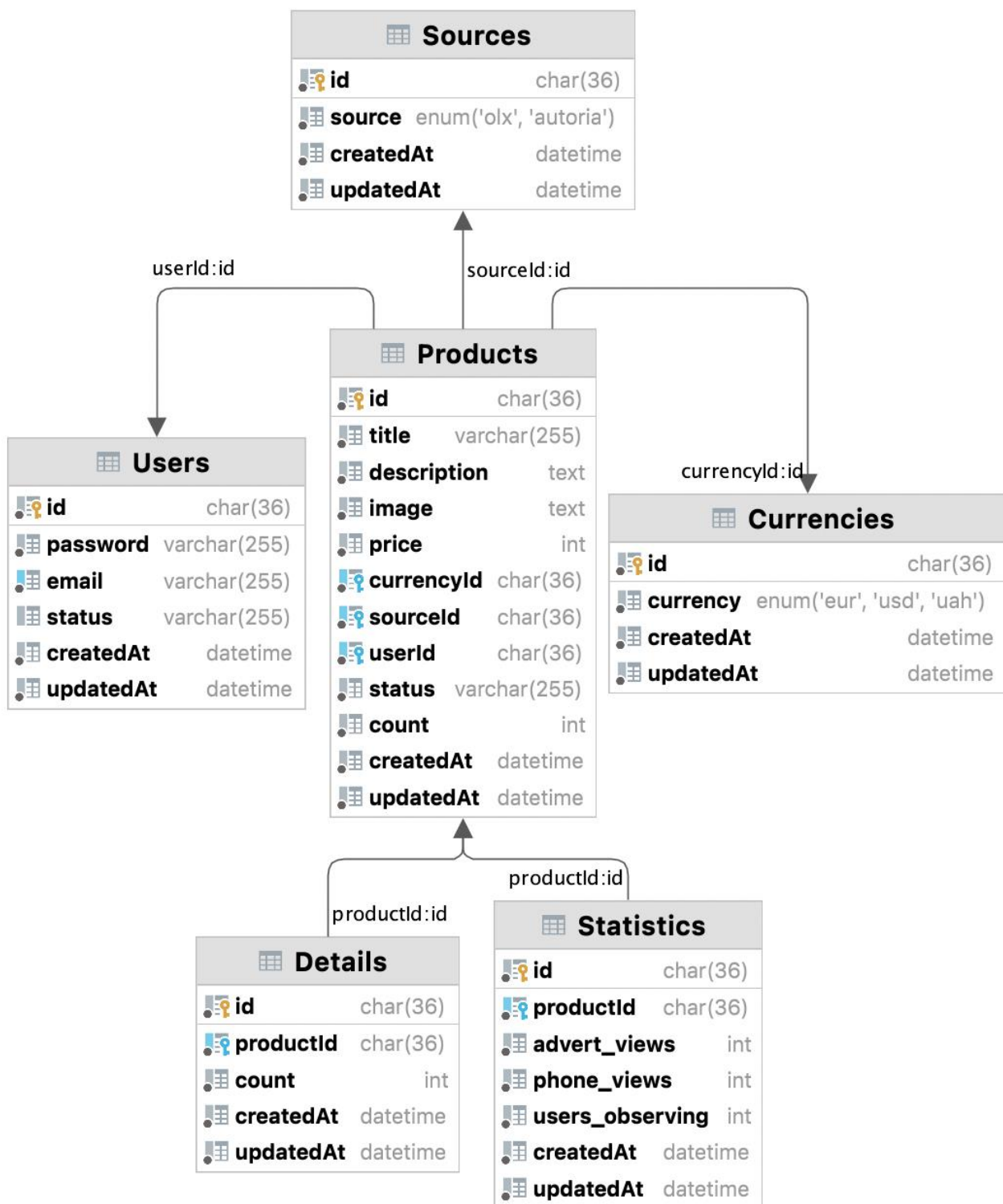


Рисунок 2.3 – Схеми бази даних

2.4. Аналіз безпеки даних

Основна інформація яку повинна бути захищена це грошові потоки та персональні дані користувача. Тобто поставлено уникнути зламування програмного забезпечення. Зламування програмного забезпечення - це удар по

репутації та іміджу компанії. Дуже неприємне у подібних подіях - розголос події. Але втрата даних, інформації про клієнтів – це прямі збитки. І розголос таких подій відбувається далеко не завжди.

Чим серйозніша компанія і відоміше її ім'я та продукти, тим істотніше бувають ризики та збитки від злому програмного забезпечення. Коли вебсайт – це імідж та репутація.

Потенційні загрози:

- злом інформаційного середовища (операційна система, вебсервер, середовище програмування, база даних);
- злом системи керування корпоративним порталом;
- зламування сторонніх вебдодатків.

Тобто серед переліченого до розробленого програмного забезпечення можна віднести лише перший пункт, оскільки корпоративного порталу не існує а, що до сторонніх веб додатків, до яких якоюсь мірою можна віднести підключені торговельні майданчики, розроблене програмне забезпечення відповідальність за цілісність їх даних не несе.

Ступінь загроз можна розділити на три рівні ризику:

- мінімальний – отримання доступу до не конфіденційної інформації, до якої не санкціоновано доступ, можливість створення косметичних проблем та перешкод у роботі проєкту;
- середній рівень – отримання часткового доступу до конфіденційної інформації, частковий обхід системи авторизації розширює повноваження;
- високий рівень – повний обхід системи авторизації, отримання необмеженого доступу до системи або додатку, можливість запуску несанкціонованих додатків, можливість перегляду або заміни конфіденційної інформації.

Розроблене програмне забезпечення не містить жоден з рівнів. Доступ до неконфіденційної інформації не можливий, оскільки роути чітко розділені на авторизованих та не авторизованих користувачів. Авторизація відбувається безпосередньо через JSON web token (JWT). Частковий або повний обхід

					КП.ІП-8233.045440.02.81	Арк.
Змін.	Арк.	№ докум.	Підп.	Дата.		38

системи авторизації також мало ймовірний через те, що при авторизації виконується трирівневе шифрування паролів, а саме SHA 512, Argon2, AES 256. Такий рівень захисту не підвладний “brute force” або “dictionary” атакам.

Для надійної передачі даних на сервер програмне забезпечення використовує захищене HTTPS з’єднання, для цього було використано зворотній проксі сервер nginx. HTTPS являє собою розширення протоколу HTTP з підтримкою шифрування. Він працює за допомогою шифрованих транспортних механізмів SSL і TLS.

2.5. Висновки до розділу

В даному розділі було змодельовано програмне забезпечення за допомогою моделі С4. Розглянуті діаграми контексту, діаграми контейнерів та діаграми компонентів. Також розглянуто та побудовано структуру бази даних.

Було розроблено архітектуру клієнтського та серверного застосунків та описано як вони взаємодіють. Також був обраний механізм для безпечного передавання даних з клієнта на сервер.

3 АНАЛІЗ ЯКОСТІ ТА ТЕСТУВАННЯ ПРОГРАМНОГО ЗАБЕЗПЕЧЕННЯ

3.1. Аналіз якості ПЗ

Тестування є важливим етапом під час процесі розробки програмного забезпечення, тим самим можна підтвердити працездатність розробленого продукту. У рамках розроблення програмного забезпечення необхідність тестування також зумовлена бізнес-логікою програмного забезпечення, а саме взаємодією з торговельними майданчиками.

Тестування програмного забезпечення - це метод перевірки відповідності фактичного програмного продукту очікуваним вимогам, також необхідний, щоб переконатися, що продукт не містить дефектів. Іншими словами це виконання попередньо визначених алгоритмів з використанням ручних або автоматизованих інструментів для оцінки одного або декількох властивостей, які цікавлять. Метою тестування є виявлення помилок, прогалин або відсутніх вимог, заданих на етапі проєктування продукту.

Сьогодні тестування – один з найбільш затратних етапів розробки, на нього виділяється від 50% до 65% загальних витрат до проєкту. На ринку є кілька способів випробування програмного забезпечення, але по суті це дослідницький процес і для дієвого випробування треба брати до уваги специфіку продукту, що розробляється. Проєкт випробування програмного забезпечення включає список тестованих компонентів, аспект проходження тесту, перелік тестових випадків та створення звіту про завершення випробування системи та звіту про промахи, що утворилися в системі.

3.2. Опис процесів тестування

3.2.1. Компоненти що тестуються

Зробимо перелік компонентів:

- авторизація користувача;
- реєстрація користувача;

- під'єднання до API торговельних майданчиків;
- перегляд всіх товарів;
- вивід статистики за товарами;
- призупинення оголошення;
- додавання товару у список проданих.

3.2.2. Підхід до тестування

Підхід до тестування складається з наступних рівнів тестування:

- модульне тестування. Виконується розробником на етапі кодування програми. Модульні тести ізолюють частину коду та перевіряють його працездатність. Одиницею для вимірювання може бути окрема функція, метод, процедура, модуль чи об'єкт. Тести реалізуються за допомогою стороннього додатку AVA [13];
- інтеграційне тестування. Для інтеграційного тестування використовуються компоненти, що вже перевірені за допомогою модульного тестування, які групуються у множини. Дані множини перевіряються відповідно до плану тестування, складеного для них, а об'єднуються вони через свої інтерфейси. Тести реалізуються за допомогою стороннього додатку AVA;
- UI тестування. Цей процес передбачає імітацію дій користувача — кліки на кнопки, переходи по посиланнях, та інші дії подібного плану. Таким чином перевіряється коректність роботи, взаємодія компонентів один з одним та зручність інтерфейсу в цілому.

3.2.3. Критерій проходження тесту

Тести вважаються пройденими у випадку у випадку не знаходження жодних помилок на усіх рівнях тестування. Також мають бути виконані усі вимоги зазначені у технічному завданні.

					КПІ.ІП-8233.045440.02.81	Арк.
Змін.	Арк.	№ докум.	Підп.	Дата.		41

3.3. Опис контрольного прикладу

Метою у контрольному прикладі є перевірка коректності роботи сценарію додавання продукту у список проданих користувачем. У таблиці 3.1 продемонстровано покроковий опис тестування, що включає дії користувача, очікуваний та реальний результати.

Таблиця 3.1 – опис контрольного прикладу

Дія продавця	Очікуваний результат	Реальний результат
Продавець підключає будь-які торговельні майданчики.	На екрані демонтується повідомлення про успішне підключення.	Продemonстровано повідомлення про успішне підключення.
Продавець заходить на сторінку з продуктами.	На екрані демонтується список усіх продуктів, які належать користувачу.	На екрані продемонстровано список усіх продуктів, які належать користувачу.
Продавець обирає продукт зі списку продуктів та натискає на кнопку «Продано».	На екрані демонтується модальне вікно у якому користувачеві пропонується вибрати кількість проданих товарів.	На екрані продемонстровано модальне вікно у якому користувачеві пропонується вибрати кількість проданих товарів.
Продавець обирає кількість проданих товарів та натискає на кнопку «Зберегти».	На екрані демонтується успішне повідомлення про збереження наданої інформації	На екрані продемонстровано успішне повідомлення про збереження даних
Продавець натискає на кнопку «Статистика».	На екрані демонтується оновлена статистика.	На екрані продемонстрована оновлена статистика.

3.4. Висновки до розділу

В цьому розділі було розглянуто тест якості програмного забезпечення, перелічені складові, що тестуються. Також було описано підхід до тестування з описом видів тестування та сформульовано аспект проходження тесту.

Був описаний контрольний приклад тестування, у якому перераховані дії продавця, і відповідні очікувані й реальні результати.

					КПІ.ІП-8233.045440.02.81	Арк.
Змін.	Арк.	№ докум.	Підп.	Дата.		43

4 ВПРОВАДЖЕННЯ ТА СУПРОВІД ПРОГРАМНОГО ЗАБЕЗПЕЧЕННЯ

4.1. Розгортання програмного забезпечення

Для розгортання програмного забезпечення необхідним аспектом є попереднє встановлення пакетного менеджера NPM та наявність Node js 16 версії. У випадку з Linux вони можуть бути встановлені за допомогою NVM [14].

Програмне забезпечення було розгорнуто за допомогою хмарного сервісу Amazon:

- Amazon EC2. Сервіс який надав віртуальну серверну платформу, системи зберігання даних та балансувальник навантаження. Було вибрано заздалегідь налаштований сервер з попередньо встановленою операційною системою Linux. Сервіс також надав можливість створювати образи або використовувати власну ОС. Також для забезпечення безпеки було обмежено доступ до серверів EC2 за IP-адресами;
- Amazon RDS. Сервіс який надав віртуальну базу даних, розташовану на виділеному сервері. При цьому платформа налаштована та оптимізована під обрану БД, а саме MySQL. Був вибраний мінімальний розмір дискового простору під базу даних – 5 Тб.

Для безперервної підтримки програми в робочому стані та для змоги перезавантажувати її без простоїв, а також забезпечувати виконання загальних завдань системного адміністрування був вибраний диспетчер процесів робочого середовища PM2 [15]. Також перевагою PM2 є якісний вивід логів так помилок якщо такі наявні.

Таким чином для першого запуску програмного забезпечення на сервері знадобиться:

- 1) розгорнути EC2 та RDS;
- 2) створити зв'язок з EC2 за допомогою протоколу SSH;
- 3) встановити Node.js та NPM;

					КПІ.ІП-8233.045440.02.81	Арк.
Змін.	Арк.	№ докум.	Підп.	Дата.		44

- 4) зклонувати код з GitHub та внести у змінні оточення RDS налаштування;
- 5) запустити програмне забезпечення за допомогою PM2.

Для оновлення програмного забезпечення на сервері знадобиться:

- 1) створити зв'язок з EC2 за допомогою протоколу SSH;
- 2) зтягнути нові дані з GitHub;
- 3) оновити за допомогою PM2.

4.2. Робота з програмним забезпеченням

При запуску веб додатка пропонується авторизуватися або зареєструватися в додатку, як показано на рисунку 4.1.

Вхід

ДАЛІ

Рисунок 4.1 – Авторизація у вебдодатку

Для того, щоб приєднати торговельні майданчики необхідно перейти на вкладку «Trading Platforms» та вибрати для підключення бажаний сервіс, як показано на рисунку 4.2.

Виберіть Торговельний Майданчик



АВТОРИЗУВАТИСЯ ЧЕРЕЗ OLX



АВТОРИЗУВАТИСЯ ЧЕРЕЗ AUTORIA

Рисунок 4.2 – Сторінка для підключення торговельних майданчиків

При підключенні сервісу OLX потрібно надати дозвіл, як показано на
рисунку 4.3.

Додаток Analytic Subsystem

Дозволити доступ до вашого аккаунту?

alexfilatov380 (alexfilatov380@gmail.com)

Додатку будуть доступні:

читання даних профілю, оголошень, повідомлень
редагування даних профілю, оголошень,
повідомлень

Ви можете змінити налаштування доступу в вашому
Профілі.

Вибравши "Дозволити", ви надаєте програмі Analytic
Subsystem доступ до свого облікового запису OLX і
дозволяєте їй діяти від вашого імені відповідно до Угоди
користувача сервісів OLX та умов використання додатку.

ДОЗВОЛИТИ

ВІДМІНА

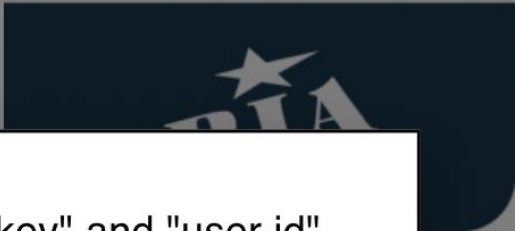
Рисунок 4.3 – Авторизація через сервіс OLX

При підключенні сервісу AutoRia необхідно ввести свої унікальні секрети, які можна знайти у своєму обліковому записі, та ввести їх у форму (рис. 4.4).

					КПІ.ІП-8233.045440.02.81	Арк.
Змін.	Арк.	№ докум.	Підп.	Дата.		47

Виберіть Торговельний Майданчик



АВТОРІА

АВТОРІА

Please provide your "api key" and "user id"

api key *

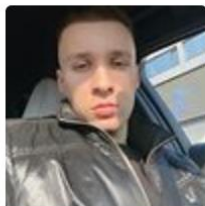
user id *

SUBMIT

Рисунок 4.4 – Авторизація через сервіс AutoRia

На рисунку 4.5 зображена сторінка профілю AutoRia де можна знайти секрети для доступу до товарів.

Мой профиль



Email	alexfilatov380@gmail.com
Ваш ID (user_id)	11267851
Api Key	9T6v4q... Показати
Зареєстрований	2022-05-18 11:15:01

[Редагувати](#)[Змінити пароль](#)

Рисунок 4.5 – Сторінка з ключами доступу до AutoRia

Після підключення торговельних майданчиків користувач зможе побачити та відфільтрувати всі свої продукти на головній сторінці вебдодатка (рис. 4.6).



Iphone 11 256gb black

Торговельний Майданчик: OLX

Ціна: 1000 USD

Продано: 12 шт.

Статус: active

Дата Створення: 05/14/2022

Термін Придатності: 06/13/2022

Статистика: 10 0 0



ПЕРЕЙТИ ДО
РЕСУРСУ



Honda CR-V 2007

Торговельний Майданчик: AutoRIA

Ціна: 10000 USD

Продано: 1 шт.

Статус: active

Дата Створення: 05/18/2022

Термін Придатності: 06/18/2022

Статистика: 353 0 2



ПЕРЕЙТИ ДО
РЕСУРСУ

Рисунок 4.6 – Демонстрація карток товарів

На сторінці статистики користувачу буде продемонстрований рейтинг товарів (рис. 4.7) та загальний прибуток (рис. 4.8).

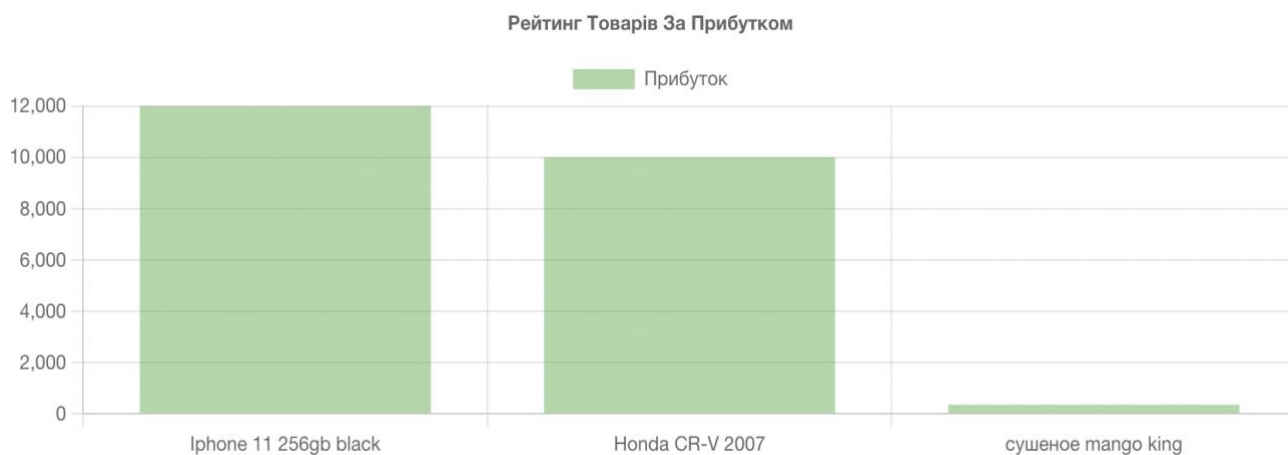


Рисунок 4.7 – Рейтинг товарів

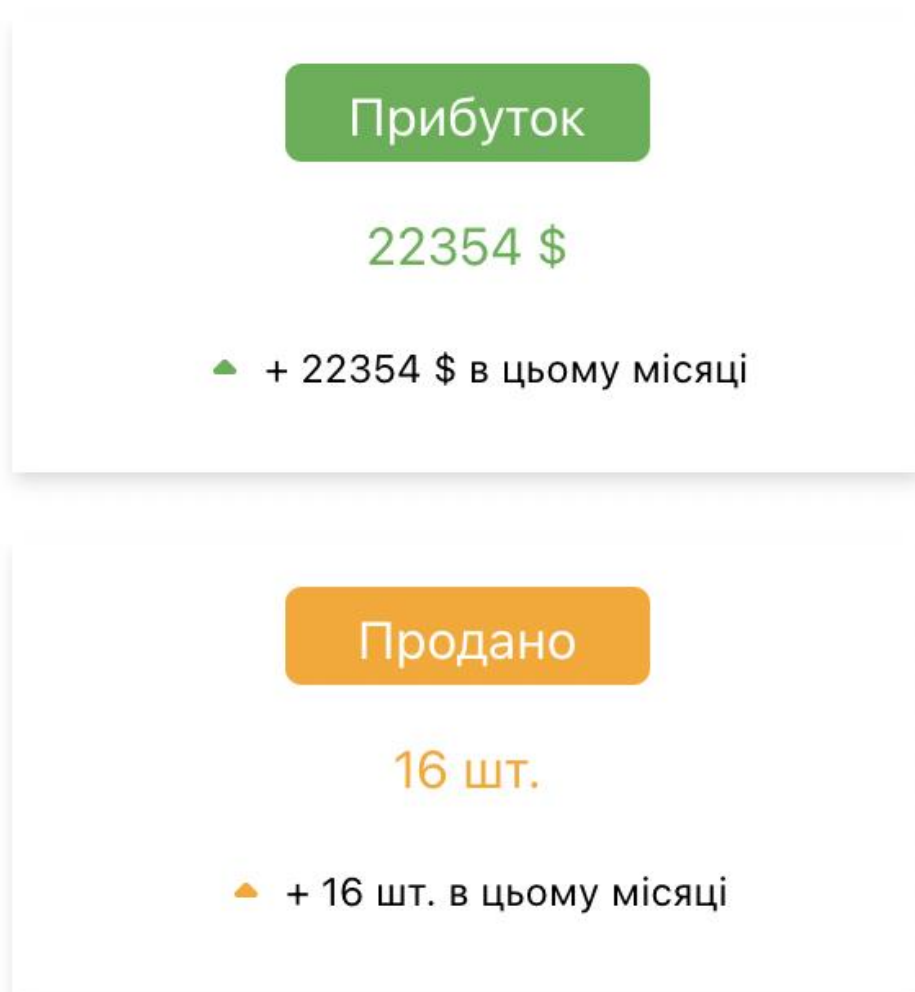


Рисунок 4.8 – Рейтинг товарів

Також користувач зможе скористатися на сторінці статистики лінійний графік по кожному з товарів в якому він зацікавлений попередньо вказавши діапазон (тиждень, місяць або рік), як показано на рисунку 4.9.

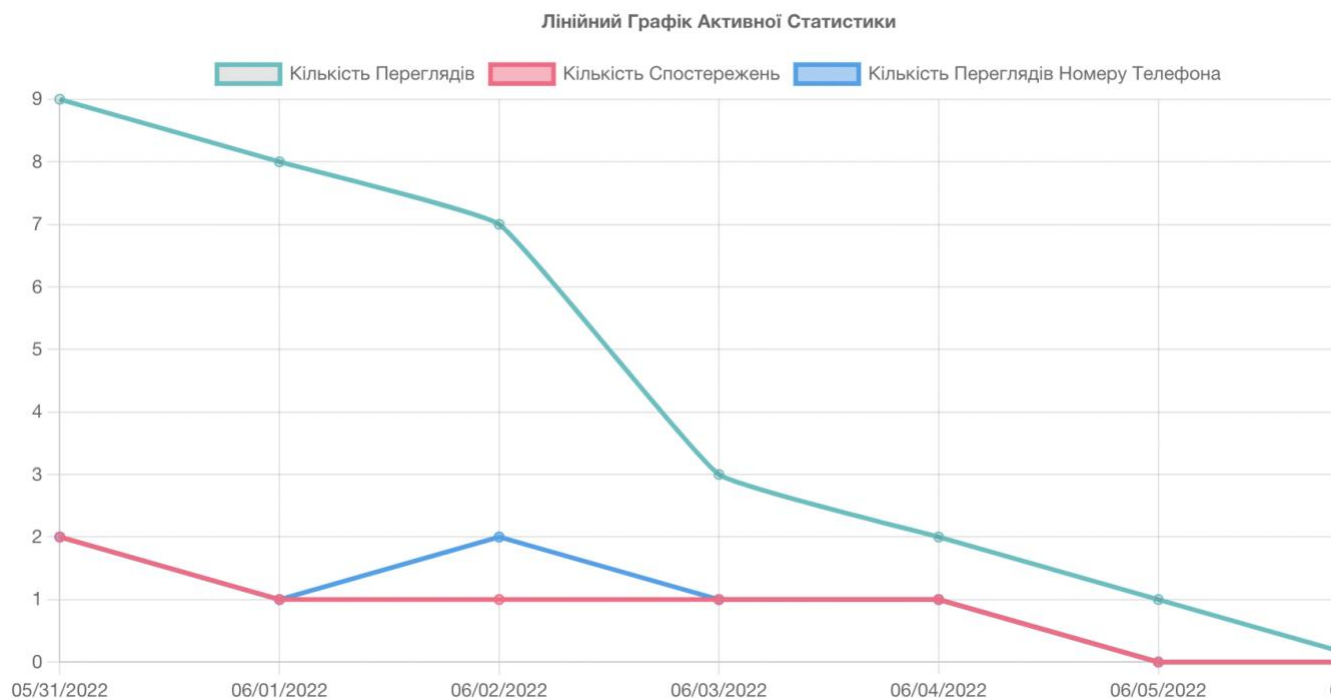


Рисунок 4.9 – Лінійний графік активності товару

Користувач має можливість переглядати активну статистику за трьома пунктами це кількість переглядів (рис. 4.10), кількість спостережень та кількість переглядів номеру телефона.

Кількість Переглядів

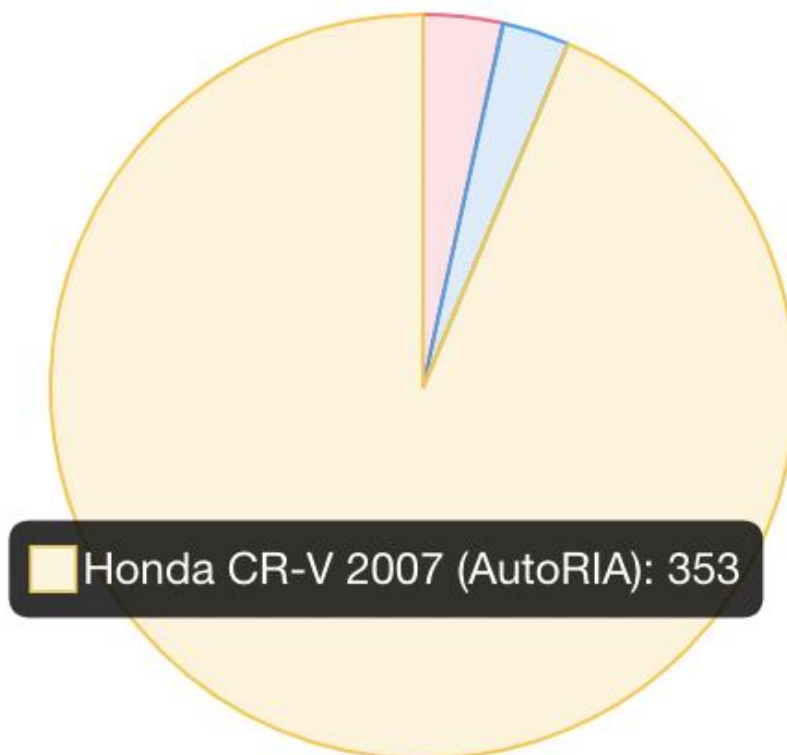
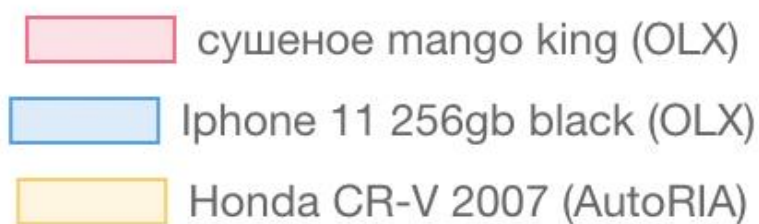


Рисунок 4.10 – Діаграма за кількістю переглядів

Щоб переглянути власний профіль треба натиснути на лівій боковій панелі кнопку «Налаштування» (є можливість зміни мови), рисунок 4.11.

Мова

Українська  ▼



Персональна Інформація

email

alexfilatov380@gmail.com

password

ВИЙТИ

Рисунок 4.11 – Профіль користувача

4.3. Висновки до розділу

Програмне забезпечення було розгорнуте на хмарному сервісі Amazon. Працездатність програмного забезпечення було проведена шляхом виконання контрольного тесту. Програмне забезпечення повністю готове для впровадження.

					КПІ.ІП-8233.045440.02.81	Арк.
Змін.	Арк.	№ докум.	Підп.	Дата.		54

ВИСНОВКИ

Було розроблено програмне забезпечення, яке функціонально дозволяє продавцю товарів організувати свій бізнес на торговельних майданчиках таких як OLX та AutoRia. Розроблена програмне забезпечення об'єднує дані торговельних майданчиків в одній системі та дозволяє користувачеві аналізувати свій прибуток та бачити статистику активних оголошень.

Був проведений аналіз аналогів програмного забезпечення, визначені їх недоліки. Розроблена архітектура розробленого програмного забезпечення. Також було визначено перелік необхідних програмних засобів для реалізації проєкту. Серед них було вибрано Node.js для сервера, Reac.js для клієнта та MySQL для бази даних.

Розроблено моделі інтерфейсу та структури даних, які необхідні для функціонування системи, описані бізнес-процеси розроблюваного програмного забезпечення з використанням моделі типу C4.

Для аналізу якості побудованого програмного забезпечення, було проведене його тестування, у наслідок чого було знайдено та усунено недоліки.

Для розгортання програмного забезпечення було обрано хмарний сервіс Amazon. Для перевірки працездатності програмного забезпечення був виконаний контрольний тест. Програмне забезпечення повністю готове для впровадження.

					КПІ.ІП-8233.045440.02.81	Арк.
Змін.	Арк.	№ докум.	Підп.	Дата.		55

СПИСОК ВИКОРИСТАНИХ ДЖЕРЕЛ

1. OLX API [електронний ресурс] URL: <https://developer.olx.ua/api/doc/>
2. AutoRia API [електронний ресурс] URL: <https://developers.ria.com/docs/>
3. Gerardus Blokdyk. JSON Web Token: Third Edition / CreateSpace Independent Publishing Platform, 2018. 134 с.
4. Carlos Santana Roldan. React 17 Design Patterns and Best Practices / Birmingham: Packt Publishing, 2021. 394 с.
5. Mario Casciaro, Luciano Mammino. Node.js Design Patterns: Design and implement production-grade Node.js applications using proven patterns and techniques / Birmingham: Packt Publishing, 2020. 660 с.
6. Zachary Shute. Advanced JavaScript: Speed up web development with the powerful features and benefits of JavaScript / Birmingham: Packt Publishing, 2019. 332 с.
7. Express.js [електронний ресурс] URL: <https://expressjs.com/>
8. Silvia Botros, Jeremy Tinley. High Performance MySQL: Proven Strategies for Operating at Scale / Newton: O'Reilly Media, 2021. 388 с.
9. Amazon [електронний ресурс] URL: <https://docs.aws.amazon.com/>
10. The C4 model for visualising software architecture [електронний ресурс] URL: <https://c4model.com/>
11. SHA-512 Hashing Algorithm [електронний ресурс] URL: <https://komodoplatform.com/en/academy/sha-512/>
12. Advanced Encryption Standard (AES) [електронний ресурс] URL: <https://www.techtarget.com/searchsecurity/definition/Advanced-Encryption-Standard/>
13. AVA [електронний ресурс] URL: <https://github.com/avajs/ava/>
14. NVM [електронний ресурс] URL: <https://ionicframework.com/docs/v3/developer-resources/using-nvm/>
15. PM2 [електронний ресурс] URL: <https://pm2.keymetrics.io/>

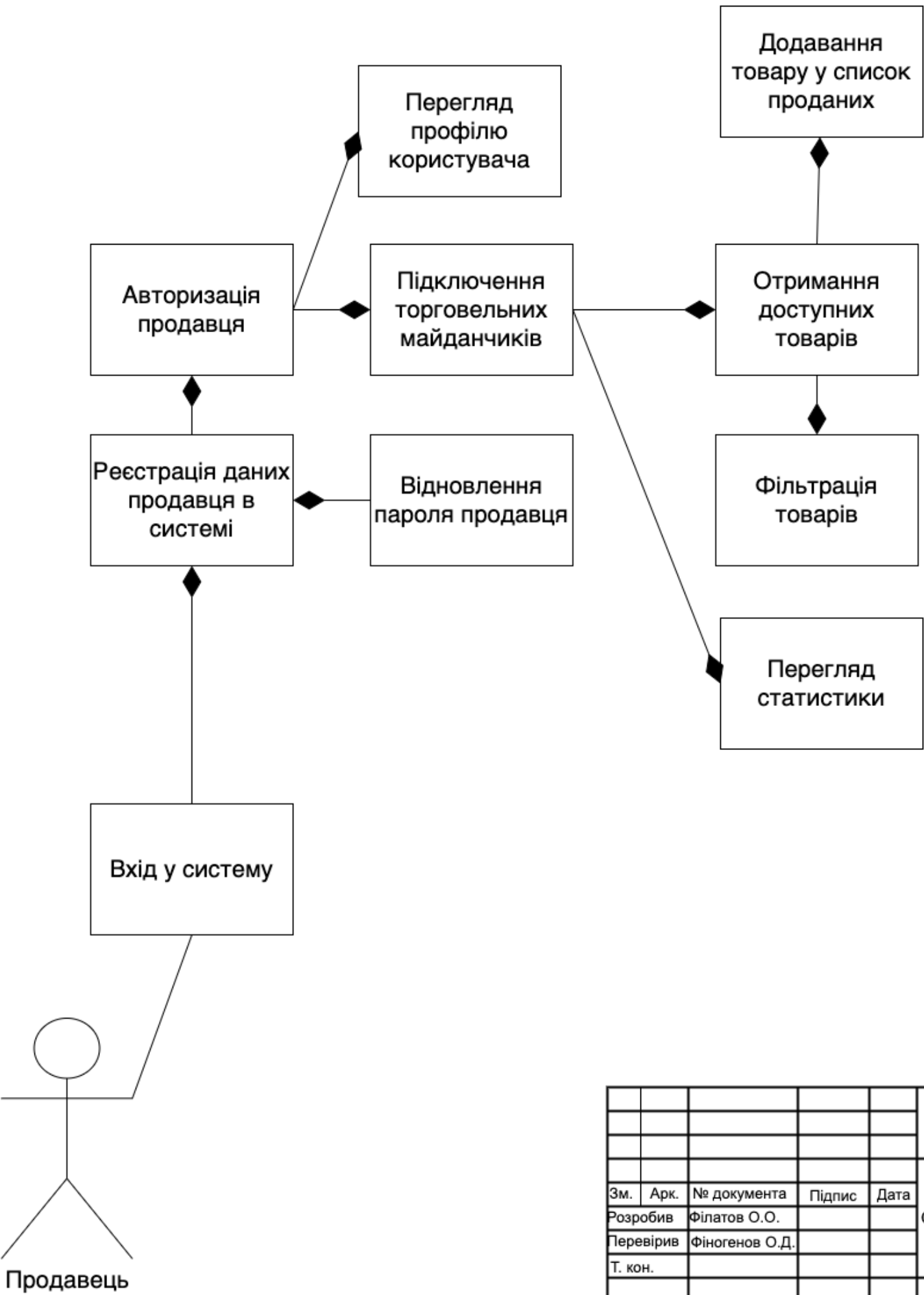
ДОДАТОК А

**Програмне забезпечення для підтримки та аналітики для продавців на
торговельних майданчиках**

**Схема структурна варіантів використання
КПІ.ІП-8233.045440.06.99**

Аркушів 1

Київ – 2022



					КПІ.ІП-8233.045440.06.99.СС			
					Схема структурна варіантів використання	Літера	Маса	Масштаб
Зм.	Арк.	№ документа	Підпис	Дата				
Розробив		Філатов О.О.						
Перевірів		Фіногенов О.Д.						
Т. кон.					Програмне забезпечення для підтримки та аналітики для продавців на торговельних майданчиках	Аркуш		Аркушів
Н. кон.		Ліщук К.І.				КПІ ім.Ігоря Сікорського Кафедра АСОІУ гр. ІП-82		
Затвердив		Жаріков Е.В.						

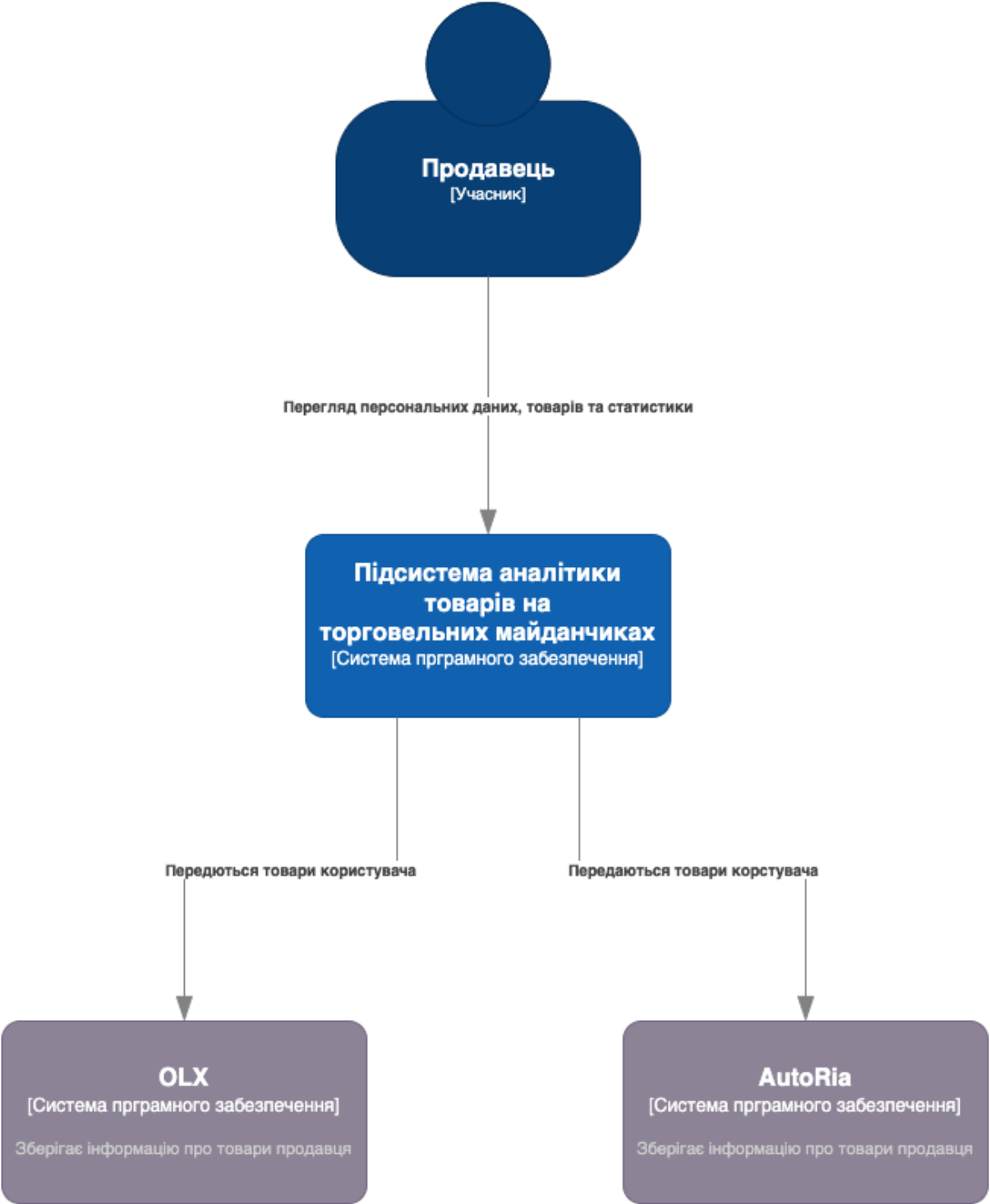
ДОДАТОК Б

Програмне забезпечення для підтримки та аналітики для продавців на
торговельних майданчиках

Схема структурна
контексту програмного
забезпечення
КПІ.ІП-8233.045440.07.99

Аркушів 1

Київ – 2022



					КПІ.ІП-8233.045440.07.99.СС						
Зм.	Арк.	№ документа	Підпис	Дата	Схема структурна контексту програмного забезпечення			Літера		Маса	Масштаб
Розробив	Філатов О.О.										
Перевішив	Фіногенов О.Д.										
Т. кон.											
					Програмне забезпечення для підтримки та аналітики для продавців на торговельних майданчиках			Аркуш		Аркушів	
Н. кон.		Ліщук К.І.						КПІ ім.Ігоря Сікорського Кафедра АСОІУ гр. ІП-82			
Затвердив		Жаріков Е.В.									

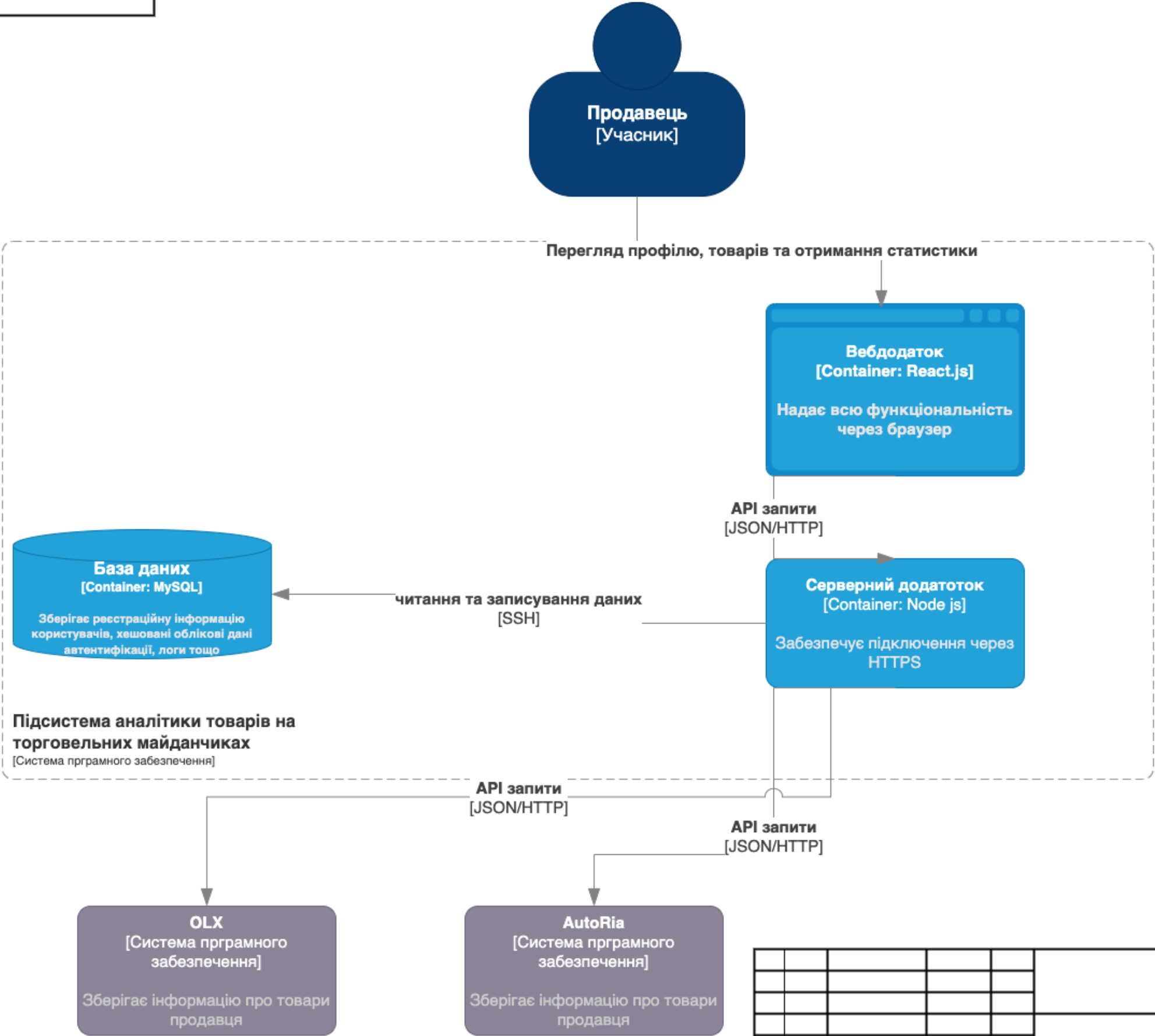
ДОДАТОК В

Програмне забезпечення для підтримки та аналітики для продавців на
торговельних майданчиках

Схема структурна
контейнерів програмного
забезпечення
КПІ.ІП-8233.045440.08.99

Аркушів 1

Київ – 2022



					КПІ.ІП-8233.045440.08.99.СС									
					Схема структурна контейнерів програмного забезпечення					Літера		Маса	Масштаб	
Зм.	Арк.	№ документа	Підпис	Дата										
Розробив	Філатов О.О.													
Перевірів	Фіногенов О.Д.													
Т. кон.					Програмне забезпечення для підтримки та аналітики для продавців на торговельних майданчиках					Аркуш		Аркушів		
Н. кон.		Лішук К.І.												
Затвердив		Жаріков Е.В.								КПІ ім.Ігоря Сікорського Кафедра АСОІУ гр. ІП-82				

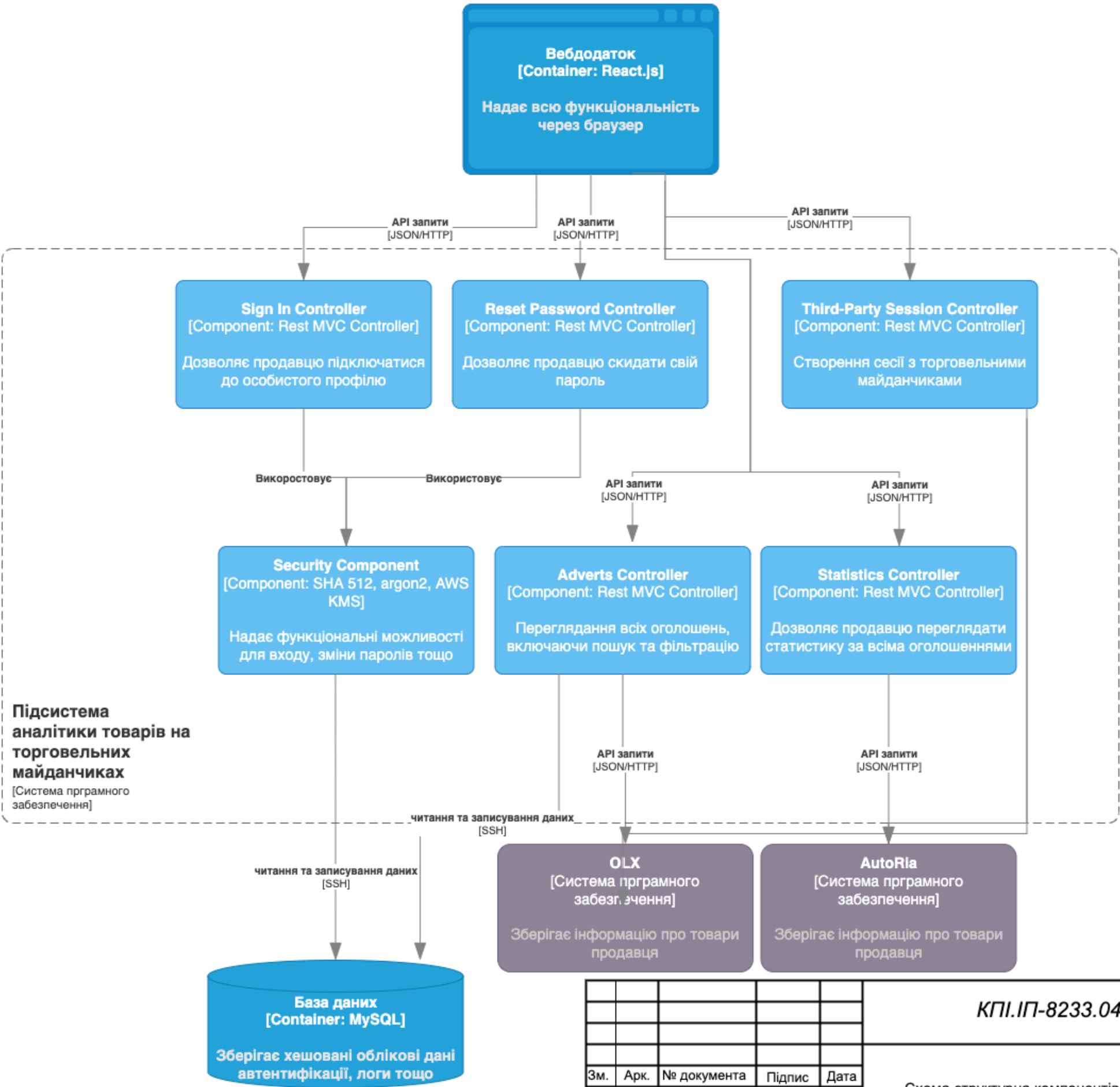
ДОДАТОК Д

Програмне забезпечення для підтримки та аналітики для продавців на
торговельних майданчиках

Схема структурна
компонентів програмного
забезпечення
КП.ІП-8233.045440.09.99

Аркушів 1

Київ – 2022



					КПІ.ІП-8233.045440.09.99.СС					
					Схема структурна компонентів програмного забезпечення	Літера		Маса	Масштаб	
Зм.	Арк.	№ документа	Підпис	Дата						
Розробив		Філатов О.О.								
Перевірив		Фіногенов О.Д.								
Т. кон.						Аркуш		Аркушів		
					Програмне забезпечення для підтримки та аналітики для продавців на торговельних майданчиках	КПІ ім.Ігоря Сікорського Кафедра АСОІУ гр. ІП-82				
Н. кон.		Ліщук К.І.								
Затвердив		Жаріков Е.В.								

Факультет інформатики та обчислювальної техніки
Кафедра інформатики та програмної інженерії

“ЗАТВЕРДЖЕНО”

Завідувач кафедри

_____ Едуард ЖАРІКОВ

“ ____ ” _____ 2022 р.

**ПРОГРАМНЕ ЗАБЕЗПЕЧЕННЯ ДЛЯ ПІДТРИМКИ ТА АНАЛІТИКИ
ДЛЯ ПРОДАВЦІВ НА ТОРГОВЕЛЬНИХ МАЙДАНЧИКАХ**

Опис програми

КПІ.ІП-8233.045440.03.13

“ПОГОДЖЕНО”

Керівник проекту:

_____ Олексій Фіногенов

Нормоконтроль:

_____ Катерина ЛІЩУК

Виконавець:

_____ Олександр ФІЛАТОВ

Київ – 2022

ТЕКСТ ПРОГРАМНОГО КОДУ

Тексти програмного коду

Програмне забезпечення для підтримки та аналітики для продавців на торговельних майданчиках

(Найменування програми (документа))

CD-R

(Вид носія даних)

40 арк, 2372 Кб

(Обсяг програми, арк., Кб)

Київ – 2022

					КПІ.ІП-8233.045440.03.13	Арк.
Змін.	Арк.	№ докум.	Підп.	Дата.		1

```

import fs      from 'fs';
import https   from 'https';
import express from 'express';
import bluebird from 'bluebird';
import cors    from 'cors';
import bodyParser from 'body-parser';
import adminRoutes from './admin/router.mjs';

const promisify = bluebird.promisifyAll;

const app = express();

let server = null;

export function init({ sequelize }) {
  app.use(cors());
  app.use(express.static('public'));
  app.use(bodyParser.urlencoded({ extended: false }));
  app.use(bodyParser.json());

  app.use('/storage', express.static('storage'));
  app.use('/api/v1', adminRoutes({ sequelize }));

  return app;
}

export function start({ appPort, secure }) {
  const securedApp = secure ? useHttps(app) : app;

  server = securedApp.listen(appPort, () => {
    console.log(`app started on port ${appPort}`);
  });

  server.closeAsync = promisify(server.close);
}

export async function stop() {
  if (!server) return;

  console.log('server was stopped');

  await server.closeAsync();
}

function useHttps(application) {
  const httpsOptions = {
    key : fs.readFileSync('./certs/localhost-key.pem'),
    cert : fs.readFileSync('./certs/localhost.pem')
  };

  return https.createServer(httpsOptions, application);
}

import jwt from 'jsonwebtoken';
import User from '../domain-model/User.mjs';
import config from '#global-config' assert {type: 'json'};

```

					КПІ.ІП-8233.045440.03.13	Арк.
Змін.	Арк.	№ докум.	Підп.	Дата.		2

```

export async function runUseCase(useCaseClass, { context = {}, params = {} }) {
  const result = await new useCaseClass({ context }).run(params);

  return result;
}

export function makeUseCaseRunner(
  useCaseClass,
  paramsBuilder,
  params,
  render = renderPromiseAsJson
) {
  return async function useCaseRunner(req, res, next) {
    let context = null;

    if (params?.withToken) {
      context = await validateJwt(req, res);
      if (!context) return;
    }

    const resultPromise = runUseCase(useCaseClass, {
      params : paramsBuilder(req, res),
      context
    });

    return render(req, res, resultPromise, next);
  };
}

async function validateJwt(req, res) {
  try {
    const token = getToken(req);

    const userData = await jwt.verify(token, config.tokenLoginKey);

    const user = await User.findByPk(userData.id);

    if (!user) {
      throw new Error('NOT_VALID_USER');
    }

    return {
      userId : user.id,
      userInstance : user
    };
  } catch (e) {
    res.status(400).send({
      error : 'WRONG_TOKEN'
    });
  }
}

export async function renderPromiseAsJson(req, res, promise) {
  try {
    const data = await promise;
  }
}

```

```

    data.status = 1;

    return res.send(data);
  } catch (error) {
    console.log(error);
    res.send({
      status : 0,
      error : {
        code : 'SERVER_ERROR',
        message : 'Please, contact your system administartor!'
      }
    });
  }
}

function getToken(req) {
  if (req.headers.authorization && req.headers.authorization.split(' ')[0] === 'Bearer') {
    return req.headers.authorization.split(' ')[1];
  }

  if (req.query && req.query.token) {
    return req.query.token;
  }

  if (req.params && req.params.token) {
    return req.params.token;
  }

  if (req.params && req.params.token) {
    return req.params.token;
  }

  if (req.get('X-AuthToken')) {
    return req.get('X-AuthToken');
  }

  return null;
}

import Base from '../././use-cases/Base.mjs';

export default class AutoriaSession extends Base {
  async validate(data = {}) {
    const rules = {
      ria_access_token : [ 'required', 'string' ],
      user_id : [ 'required', 'string' ]
    };

    return this.doValidation(data, rules);
  }

  async execute({ ria_access_token, user_id }) {
    let response;
    try {
      response = await
        fetch(`https://developers.ria.com/auto/used/autos/ids?api_key=${ria_access_token}&user_id=${user_id}`

```

					КПІ.ІП-8233.045440.03.13	Арк.
Змін.	Арк.	№ докум.	Підп.	Дата.		4

```

    }, {
        method : 'GET',
        headers : {
            'Content-Type' : 'application/json'
        }
    });
} catch (e) {
    throw e;
}

let result = await response.json();

if (result?.active) {
    result = {
        ria_access_token,
        ria_user_id : user_id
    };
} else {
    result = { error: result?.error?.code || 'AUTORIA_SESSION_FAILED' };
}

return {
    data : { ...result }
};
}
}

```

```
import Base from '../use-cases/Base.mjs';
```

```

export default class CreateSession extends Base {
    async validate(data = {}) {
        const rules = {
            grant_type : [ 'required', 'string' ],
            client_id   : [ 'required', 'string' ],
            client_secret : [ 'required', 'string' ],
            scope       : [ 'string', { default: 'v2 read write' } ],
            code        : [ 'required', 'string' ],
            redirect_uri : [ 'required', 'string' ]
        };

```

```

        return this.doValidation(data, rules);
    }

```

```

    async execute(data) {
        let response;

        try {
            response = await fetch('https://www.olx.ua/api/open/oauth/token', {
                method : 'POST',
                headers : {
                    'Content-Type' : 'application/json'
                },
                body : JSON.stringify(data)
            });
        } catch (e) {
            throw e;
        }
    }

```

					КПІ.ІП-8233.045440.03.13	Арк.
Змін.	Арк.	№ докум.	Підп.	Дата.		5


```

    }

    const result = await response.json();

    return {
      data: { ...result }
    };
  }
}

import fetch from 'node-fetch';
import Base from '../use-cases/Base.mjs';
import { dumpAdvert } from '../utils/dumps.mjs';
import Product from '../domain-model/Product.mjs';

export default class GetAdverts extends Base {
  async validate(data = {}) {
    const rules = {
      olx_access_token: [ 'string' ],
      ria_access_token: [ 'string' ],
      autoriaEnabled: [ 'boolean' ],
      olxEnabled: [ 'boolean' ],
      ria_user_id: [ 'string' ]
    };

    return this.doValidation(data, rules);
  }

  async execute({
    olxEnabled,
    autoriaEnabled,
    olx_access_token,
    ria_access_token,
    ria_user_id
  }) {
    const adverts = [];
    const sourceCount = {
      OLX: 0,
      AutoRIA: 0
    };

    if (olxEnabled) {
      try {
        const response = await fetch('https://www.olx.ua/api/partner/adverts', {
          method: 'GET',
          headers: {
            'Content-Type': 'application/json',
            'Authorization': `Bearer ${olx_access_token}`,
            'Version': '2.0'
          }
        });

        const olxAdverts = await response.json();

        for (const advert of olxAdverts.data) {

```

					КПІ.ІП-8233.045440.03.13	Арк.
Змін.	Арк.	№ докум.	Підп.	Дата.		6

```

const res = await fetch(`https://www.olx.ua/api/partner/adverts/${advert.id}/statistics`, {
  method : 'GET',
  headers : {
    'Content-Type' : 'application/json',
    'Authorization' : `Bearer ${olx_access_token}`,
    'Version'      : '2.0'
  }
});

advert.statistics = await res.json();
}

const dumpOlxAdverts = olxAdverts.data.map(advert => dumpAdvert({ advert, olxEnabled: true
}));

for (const advert of dumpOlxAdverts) {
  const product = await Product.findOne({
    where : {
      title : advert.title
    }
  });

  advert.soldCount = product?.count || 0;
}

adverts.push(...dumpOlxAdverts);
sourceCount.OLX = dumpOlxAdverts.length;
} catch (e) {
  throw e;
}
}

if (autoriaEnabled) {
  try {
    const response = await
fetch(`https://developers.ria.com/auto/used/autos/ids?api_key=${ria_access_token}&user_id=${ria_user
_id}`, {
  method : 'GET',
  headers : {
    'Content-Type' : 'application/json'
  }
});

const advertIds = await response.json();

if (!advertIds.active) {
  throw advertIds;
}

const advertActiveIds = advertIds.active.moderated;

const advertsInfo = [];

for (const advertId of advertActiveIds) {
  const resAdvertInfo = await
fetch(`https://developers.ria.com/auto/info?auto_id=${advertId}&api_key=${ria_access_token}`, {
    method : 'GET',

```

					КПІ.ІП-8233.045440.03.13	Арк.
Змін.	Арк.	№ докум.	Підп.	Дата.		7

```

        headers : {
            'Content-Type' : 'application/json'
        }
    });

    const resStatistic = await
fetch(`https://developers.ria.com/auto/used/autos/${advertId}/statistic?api_key=${ria_access_token}&us
er_id=${ria_user_id}`, {
    method : 'GET',
    headers : {
        'Content-Type' : 'application/json'
    }
});
    const advertInfo = await resAdvertInfo.json();
    const statistics = await resStatistic.json();

    advertsInfo.push({ ...advertInfo, statistics, id: advertId });
}

const dumpRiaAdverts = advertsInfo.map(advert => dumpAdvert({ advert, autoriaEnabled: true
}));

for (const advert of dumpRiaAdverts) {
    const product = await Product.findOne({
        where : {
            title : advert.title
        }
    });

    advert.soldCount = product?.count || 0;
}

adverts.push(...dumpRiaAdverts);
sourceCount.AutoRIA = dumpRiaAdverts.length;
} catch (e) {
    throw e;
}
}

return {
    data : {
        adverts,
        sourceCount
    }
};
}
}

```

```

import Base from '../use-cases/Base.mjs';
import Product from '../domain-model/Product.mjs';
import Statistic from '../domain-model/Statistic.mjs';
import Currency from '../domain-model/Currency.mjs';
import Source from '../domain-model/Source.mjs';
import Detail from '../domain-model/Detail.mjs';
import {Op} from "sequelize";

```

					КПІ.ІП-8233.045440.03.13	Арк.
Змін.	Арк.	№ докум.	Підп.	Дата.		8

```

import {dumpSoldStatistics} from '../utils/dumps.mjs';

export default class GetSoldStatistics extends Base {
  async execute() {

    const date = new Date();
    const firstDay = new Date(date.getFullYear(), date.getMonth(), 1);

    const products = await Product.findAll({
      include: [Currency, Source, Statistic, {
        model: Detail,
        where: {
          createdAt: {[Op.gte]: firstDay}
        },
        required: false
      }],
      where: {
        userId: this.context.userId
      }
    })

    const sortedProducts = products.map(el => ({
      ...el,
      total: el.count * el.price
    })).sort((a, b) => b.total - a.total);

    const result = sortedProducts.map(dumpSoldStatistics)

    const statistics = {
      totalCount: 0,
      totalPrice: 0
    };
    for (const val of result) {
      val.forPeriod = {};
      val.forPeriod.totalCount = val.details.reduce((a, b) => a + (b['count'] || 0), 0);
      val.forPeriod.totalPrice = val.forPeriod.totalCount * val.price;

      statistics.totalCount = statistics.totalCount + val.count;
      statistics.totalPrice = statistics.totalPrice + val.count * val.price;
    }

    return {
      data: {
        products: result,
        statistics,
        forPeriod: {
          totalCount: result.reduce((a, b) => a + (b?.forPeriod['totalCount'] || 0), 0),
          totalPrice: result.reduce((a, b) => a + (b?.forPeriod['totalPrice'] || 0), 0),
        }
      }
    };
  }
}

import crypto from 'crypto';
import Base from '../use-cases/Base.mjs';

```

					КПІ.ІП-8233.045440.03.13	Арк.
Змін.	Арк.	№ докум.	Підп.	Дата.		9

```

import Product from '../domain-model/Product.mjs';
import Statistic from '../domain-model/Statistic.mjs';
import Currency from '../domain-model/Currency.mjs';
import Source from '../domain-model/Source.mjs';
import Detail from '../domain-model/Detail.mjs';

export default class MarkAsSold extends Base {
  async validate(data = {}) {
    const rules = {
      advert: [ 'required' ],
      cnt : [ 'integer' ]
    };

    return this.doValidation(data, rules);
  }

  async execute({
    advert,
    cnt
  }) {
    const product = await Product.findOne({
      where: {
        title : advert.title,
        userId : this.context.userId
      },
      include : [ Statistic ]
    });

    const productId = product?.id || crypto.randomUUID();

    if (product) {
      await product.update({
        count : product.count + cnt
      });

      await Statistic.update({
        advert_views : product.Statistic.advert_views + (advert.statistics.advert_views -
product.Statistic.advert_views),
        phone_views : product.Statistic.phone_views + (advert.statistics.phone_views -
product.Statistic.phone_views),
        users_observing : product.Statistic.users_observing + (advert.statistics.users_observing -
product.Statistic.users_observing)

      }, {
        where: {
          productId : product.id
        }
      });
    } else {
      const source = await Source.findOne({ where: { source: advert.source } });
      const currency = await Currency.findOne({ where: { currency: advert.currency } });

      await Statistic.create({
        ...advert.statistics,
        Product: {
          id : productId,
          title : advert.title,

```

					КПІ.ІП-8233.045440.03.13	Арк.
						10
Змін.	Арк.	№ докум.	Підп.	Дата.		

```

        description : advert.description,
        image       : advert.image,
        price       : advert.price,
        status      : 'SOLD',
        sourceId    : source.id,
        currencyId  : currency.id,
        count       : cnt,
        userId      : this.context.userId
    }
}, {
    include : [ {
        model : Product
    } ]
});
}

await Detail.create({
    productId,
    count : cnt
});

return {
    data : {
        status : 1
    }
};
}
}

import fetch from 'node-fetch';
import Base from '../use-cases/Base.mjs';
import { dumpAdvert } from '../utils/dumps.mjs';

export default class CloneAdvert extends Base {
    async validate(data = {}) {
        const rules = {
            olx_access_token : [ 'string' ],
            ria_access_token : [ 'string' ],
            advert            : [ 'required' ],
            ria_user_id       : [ 'string' ],
            cnt               : [ 'required', 'integer' ]
        };

        return this.doValidation(data, rules);
    }

    async execute({
        advert,
        cnt,
        olx_access_token,
        ria_access_token,
        ria_user_id
    }) {
        if (advert.source === 'OLX') {
            const sourceData = {

```

```

    title      : advert.title,
    description : advert.description,
    category_id : advert.sourceData.category_id,
    advertiser_type : advert.sourceData.advertiser_type,
    contact     : advert.sourceData.contact,
    location    : advert.sourceData.location,
    images      : advert.sourceData.images,
    price       : advert.sourceData.price,
    attributes   : advert.sourceData.attributes?.map(el => {
        delete el.values;

        return el;
    })
  });

  for (let i = 1; i <= cnt; i++) {
    sourceData.description = `${advert.description} ${i}`;

    console.log(sourceData);
    try {
      const res = await fetch('https://www.olx.ua/api/partner/adverts', {
        method : 'POST',
        headers : {
          'Content-Type' : 'application/json',
          'Authorization' : `Bearer ${olx_access_token}`,
          'Version'      : '2.0'
        },
        body : JSON.stringify(sourceData)
      });

      const x = await res.json();

    } catch (e) {
      console.log(e);
      console.log(e?.error?.validation);
      throw e;
    }
  }

  return {
    data : {
      status : 1
    }
  };
}

```

```
import Sequelize from 'sequelize';
```

```

class Base extends Sequelize.Model {
  static init(sequelize, options = {}) {
    if (this.generateSchema) {
      this.generateSchema();
    }
  }
}

```

					КПІ.ІП-8233.045440.03.13	Арк.
Змін.	Арк.	№ докум.	Підп.	Дата.		12

```

    super.init(this.modelSchema, {
      tableName: this.tableName,
      ...options,
      sequelize,
      ...this.options
    });
  }

  static initRelationsAndHooks() {
    if (this.initRelations) this.initRelations();
    if (this.initHooks) this.initHooks();
  }
}

export default Base;

import { DataTypes as DT } from 'sequelize';
import Base from './Base.mjs';

class Currency extends Base {
  static tableName = 'Currencies';

  static generateSchema() {
    this.modelSchema = {
      id: { type: DT.UUID, defaultValue: DT.UUIDV4, primaryKey: true },
      currency: DT.ENUM('EUR', 'USD', 'UAH'),

      createdAt: { type: DT.DATE, allowNull: false },
      updatedAt: { type: DT.DATE, allowNull: false }
    };

    return this.modelSchema;
  }

  static initRelations() {
  }
}

export default Currency;

import { DataTypes as DT } from 'sequelize';
import Base from './Base.mjs';
import Product from './Product.mjs';

class Detail extends Base {
  static tableName = 'Details';

  static generateSchema() {
    this.modelSchema = {
      id: { type: DT.UUID, defaultValue: DT.UUIDV4, primaryKey: true },
      count: DT.INTEGER,
      productId: { type: DT.UUID, allowNull: false },

      createdAt: { type: DT.DATE, allowNull: false },
      updatedAt: { type: DT.DATE, allowNull: false }
    };
  }
}

```



```

};

return this.modelSchema;
}

static initRelations() {
  this.belongsTo(Product, { foreignKey: 'productId' });
}
}

export default Detail;

import { DataTypes as DT } from 'sequelize';
import Base from './Base.mjs';
import Statistic from './Statistic.mjs';
import Source from './Source.mjs';
import Currency from './Currency.mjs';
import Detail from './Detail.mjs';
import User from './User.mjs';

class Product extends Base {
  static tableName = 'Products';

  static generateSchema() {
    this.modelSchema = {
      id : { type: DT.UUID, defaultValue: DT.UUIDV4, primaryKey: true },
      title : DT.STRING,
      description : DT.TEXT,
      image : DT.TEXT,
      price : DT.INTEGER,
      currencyId : { type: DT.UUID, allowNull: false },
      sourceId : { type: DT.UUID, allowNull: false },
      userId : { type: DT.UUID, allowNull: false },
      status : DT.STRING,
      count : DT.INTEGER,

      createdAt : { type: DT.DATE, allowNull: false },
      updatedAt : { type: DT.DATE, allowNull: false }
    };

    return this.modelSchema;
  }

  static initRelations() {
    this.hasOne(Statistic);
    this.hasMany(Detail);
    this.belongsTo(Source, { foreignKey: 'sourceId' });
    this.belongsTo(Currency, { foreignKey: 'currencyId' });
    this.belongsTo(User, { foreignKey: 'userId' });
  }
}

export default Product;

import { DataTypes as DT } from 'sequelize';
import Base from './Base.mjs';

```

					КПІ.ІП-8233.045440.03.13	Арк.
Змін.	Арк.	№ докум.	Підп.	Дата.		14

```

class Source extends Base {
  static tableName = 'Sources';

  static generateSchema() {
    this.modelSchema = {
      id : { type: DT.UUID, defaultValue: DT.UUIDV4, primaryKey: true },
      source : DT.ENUM('OLX', 'AutoRia'),

      createdAt : { type: DT.DATE, allowNull: false },
      updatedAt : { type: DT.DATE, allowNull: false }
    };

    return this.modelSchema;
  }

  static initRelations() {

  }
}

export default Source;

import { DataTypes as DT } from 'sequelize';
import Base from './Base.mjs';
import Product from './Product.mjs';

class Statistic extends Base {
  static tableName = 'Statistics';

  static generateSchema() {
    this.modelSchema = {
      id : { type: DT.UUID, defaultValue: DT.UUIDV4, primaryKey: true },
      advert_views : DT.INTEGER,
      phone_views : DT.INTEGER,
      users_observing : DT.INTEGER,
      productId : { type: DT.UUID, allowNull: false },

      createdAt : { type: DT.DATE, allowNull: false },
      updatedAt : { type: DT.DATE, allowNull: false }
    };

    return this.modelSchema;
  }

  static initRelations() {
    this.belongsTo(Product, { foreignKey: 'productId' });
  }
}

export default Statistic;

import { DataTypes as DT } from 'sequelize';
import Base from './Base.mjs';

class User extends Base {

```

					КПІ.ІП-8233.045440.03.13	Арк.
Змін.	Арк.	№ докум.	Підп.	Дата.		15

```

static tableName = 'Users';

static generateSchema() {
  this.modelSchema = {
    id : { type: DT.UUID, defaultValue: DT.UUIDV4, primaryKey: true },
    password : DT.STRING,
    email : DT.STRING,
    status : DT.ENUM('UNVERIFIED', 'VERIFIED'),

    createdAt : { type: DT.DATE, allowNull: false },
    updatedAt : { type: DT.DATE, allowNull: false }
  };

  return this.modelSchema;
}

static initRelations() {

}

static async createUser(params) {
  const errors = {};
  const email = await this.findOne({ where: { email: params.email } });

  if (email) {
    errors.email = 'EMAIL_BUSY';
  }

  if (Object.keys(errors).length) {
    throw errors;
  }

  return this.create({ ...params });
}
}

export default User

import cls          from 'cls-hooked';
import Sequelize    from 'sequelize';
import User         from './User.mjs';
import Product from './Product.mjs';
import Source from './Source.mjs';
import Currency from './Currency.mjs';
import Statistic from './Statistic.mjs';
import Detail from './Detail.mjs';

const namespace = cls.createNamespace('sequelize');

Sequelize.useCLS(namespace);

export function initModels(dbConfig) {
  const { database, username, password, dialect, host, port } = dbConfig.development;

  const sequelize = new Sequelize(database, username, password, {

```

```

    host,
    port,
    dialect,
    logging : false
  });

  const models = {
    User,
    Product,
    Source,
    Currency,
    Statistic,
    Detail
  };

  Object.values(models).forEach(model => model.init(sequelize));
  Object.values(models).forEach(model => model.initRelationsAndHooks(sequelize));

  return {
    ...models,
    sequelize
  };
}

import LIVR from 'livr';
import { Exception } from './Exception.mjs';

import '../utils/livr.mjs';

export default class UseCaseBase {
  static sequelizeInstanse = null;

  constructor({ context }) {
    this.context = context;
  }

  static setSequelizeInstanse(sequelize) {
    UseCaseBase.sequelizeInstanse = sequelize;
  }

  static setAppConfig(config) {
    UseCaseBase.appConfig = config;
  }

  async run(args) {
    return UseCaseBase.sequelizeInstanse.transaction(async () => {
      let result = null;

      let cleanParams = args;

      if (this?.validate) cleanParams = await this.validate(args);

      result = this.execute(cleanParams);

      return result;
    });
  }
}

```

					КПІ.ІП-8233.045440.03.13	Арк.
Змін.	Арк.	№ докум.	Підп.	Дата.		17

```

    });
}

async doValidation(data, rules) {
    const validator = new LIVR.Validator(rules).prepare();
    const result = validator.validate(data);

    if (!result) {
        const exception = new Exception({
            code : 'FORMAT_ERROR',
            fields : validator.getErrors()
        });

        throw exception;
    }

    return result;
}

import uniq      from 'lodash/uniq.js';
import LIVR      from 'livr';
import util      from 'livr/lib/util.js';
import Validator from 'livr/lib/Validator.js';
import extraRules from 'livr-extra-rules';

function isObject(obj) {
    const prototype = Object.getPrototypeOf(obj);

    return Object(obj) === obj && (prototype === Object.prototype || !prototype);
}

function validateRules(value, rules, outputArr) {
    const validator = new LIVR.Validator({
        value : rules
    });
    const result = validator.validate({ value });

    if (!result) {
        return validator.getErrors().value;
    }

    outputArr.push(result.value);

    return;
}

const defaultRules = {
    ...extraRules,
    'list_min_length'(minLength) {
        return (value, params, outputArr) => {
            if (util.isNoValue(value)) return;
            if (!Array.isArray(value)) return 'FORMAT_ERROR';

            if (value.length < minLength) return 'TOO_SHORT';
        }
    }
}

```

```

        outputArr.push(value);
    };
},
'ne'(notAllowedValue) {
    return (value, params, outputArr) => {
        if (util.isNoValue(value)) return;
        if (!util.isPrimitiveValue(value)) return 'FORMAT_ERROR';

        if (`${value}` === `${notAllowedValue}`) {
            return 'NOT_ALLOWED_VALUE';
        }

        outputArr.push(notAllowedValue);

        return;
    };
},
'phone'() {
    return (value, params, outputArr) => {
        if (value) {
            const digits = value.replace(/^\+/, '');

            if (!digits.match(/^(380)\d*$/)) return 'WRONG_PHONE_FORMAT';
            if (digits.length !== 12) return 'WRONG_PHONE_LENGTH';

            outputArr.push(digits);
        }
    };
},
'not_null'() {
    return (value) => {
        if (value === null) return 'CANNOT_BE_NULL';

        return;
    };
},
'nullable'() {
    return (value, params, outputArr) => {
        if (util.isNoValue(value)) return;
        if (!util.isPrimitiveValue(value)) return 'FORMAT_ERROR';

        if ([ null, 'null' ].indexOf(value) >= 0) {
            outputArr.push(null);
        }
    };
},
'password'() {
    return (value) => {
        if (value) {
            if (value.match(/^[^A-Za-z\d!"#$%&'()*+,-./:;<=>?@[\\]^_`{|}~]+/)) {
                return 'PASSWORD_CONTAINS_UNSUPPORTED_SYMBOLS';
            }

            if (!value.match(/^(?=.*[a-z])(?=.*[A-Z\d!"#$%&'()*+,-./:;<=>?@[\\]^_`{|}~}{6,}$)/)) {
                return 'PASSWORD_NOT_STRONG_ENOUGH';
            }
        }
    };
}

```

					КП.ІП-8233.045440.03.13	Арк.
Змін.	Арк.	№ докум.	Підп.	Дата.		19

```

        return;
    }
};
},
'json'() {
    return (value, params, outputArr) => {
        if (!value) return;

        let data = null;

        try {
            data = JSON.parse(value);
        } catch (e) {
            return;
        }

        outputArr.push(data);
    };
},
'required_if_present'(queryKey) {
    return (value, params) => {
        if (queryKey && params[queryKey] && util.isNoValue(value)) {
            return 'REQUIRED';
        }

        return;
    };
},
'unique_list'() {
    return list => {
        if (list === undefined || list === '') return;
        if (!Array.isArray(list)) return 'FORMAT_ERROR';
        if (uniq(list).length !== list.length) return 'ITEMS_NOT_UNIQUE';

        return;
    };
},
'date_greater_than_field'(field) {
    return (value, params) => {
        if (util.isNoValue(value)) return;
        if (!util.isPrimitiveValue(value)) return 'FORMAT_ERROR';

        const epochField = Date.parse(params[field]);
        const epoch = Date.parse(value);

        if (!epochField && epochField !== 0) return;
        if (!epoch && epoch !== 0) return 'WRONG_DATE';

        const thisDate = new Date(epoch);
        const comparedDate = new Date(epochField);

        if (thisDate <= comparedDate) return 'DATE_TOO_LOW';

        return;
    };
},

```

					КПІ.ІП-8233.045440.03.13	Арк.
Змін.	Арк.	№ докум.	Підп.	Дата.		20

```

'date_less_then_field'(field) {
  return (value, params) => {
    if (util.isNoValue(value)) return;
    if (!util.isPrimitiveValue(value)) return 'FORMAT_ERROR';

    const epochField = Date.parse(params[field]);
    const epoch = Date.parse(value);

    if (!epochField && epochField !== 0) return;
    if (!epoch && epoch !== 0) return 'WRONG_DATE';

    const thisDate = new Date(epoch);
    const comparedDate = new Date(epochField);

    if (thisDate >= comparedDate) return 'DATE_TOO_HIGH';

    return;
  };
},
/*
 * Fix [Object: null prototype] validation error
 * @see https://stackoverflow.com/questions/56298481/how-to-fix-object-null-prototype-title-product
 */
'nested_object'(livr, ruleBuilders) {
  const validator = new Validator(livr).registerRules(ruleBuilders).prepare();

  return (nestedObject, params, outputArr) => {
    if (util.isNoValue(nestedObject)) return;
    if (!isObject(nestedObject)) return 'FORMAT_ERROR';

    if (!util.isObject(nestedObject) && isObject(nestedObject)) {
      Object.setPrototypeOf(nestedObject, Object.prototype);
    }

    const result = validator.validate(nestedObject);

    if (result) {
      outputArr.push(result);

      return;
    }

    return validator.getErrors();
  };
},
'rules_if'(fields) {
  return (value, params, outputArr) => {
    for (const field in fields.if) {
      if (validateRules(params[field], fields.if[field], [])) return;
    }

    return validateRules(value, fields.rules, outputArr);
  };
}
};

LIVR.Validator.registerDefaultRules(defaultRules);

```

					КПІ.ІП-8233.045440.03.13	Арк.
Змін.	Арк.	№ докум.	Підп.	Дата.		21


```

import nodemailer from 'nodemailer';
import config from '#global-config' assert {type: 'json'};

const transporter = nodemailer.createTransport({
  service: 'gmail',
  auth: {
    user: config.email,
    pass: config.emailPassword
  }
});

export default async function sendToVerify(email, url, subject, txt) {
  const mailOptions = {
    from: config.app.email,
    to: email,
    subject,
    text: `${txt}${url}`
  };

  await transporter.sendMail(mailOptions);
}

import React, {useEffect, useState} from "react";
import {useTranslation} from 'react-i18next'
import {useDispatch, useSelector} from "react-redux";
import * as rr from "react-redux";
import {createRiaSession, createSession, getAdverts, cloneAdverts, markAsSold} from
"./../redux/modules/marketplaces";
import moment from 'moment';
import LocalPhoneOutlinedIcon from '@mui/icons-material/LocalPhoneOutlined';
import RemoveRedEyeOutlinedIcon from '@mui/icons-material/RemoveRedEyeOutlined';
import FavoriteBorderOutlinedIcon from '@mui/icons-material/FavoriteBorderOutlined';
import CheckCircleOutlineRoundedIcon from '@mui/icons-material/CheckCircleOutlineRounded';
import DeleteOutlineRoundedIcon from '@mui/icons-material/DeleteOutlineRounded';
import {
  Card,
  CardActions,
  CardContent,
  CardMedia,
  Button,
  Typography,
  Box,
  ButtonBase,
  FormControl,
  InputLabel,
  Select,
  OutlinedInput,
  MenuItem,
  Autocomplete,
  TextField,
  Modal,
  Dialog,
  DialogTitle,
  DialogContent,
  DialogContentText,

```

					КПІ.ІП-8233.045440.03.13	Арк.
Змін.	Арк.	№ докум.	Підп.	Дата.		22

```

DialogActions
} from "@mui/material";
import SearchIcon from '@mui/icons-material/Search';
import {useNavigate} from "react-router-dom";

const Tr = useTranslation;

const style = {
  position: 'absolute',
  top: '50%',
  left: '50%',
  transform: 'translate(-50%, -50%)',
  width: 400,
  bgcolor: 'background.paper',
  border: '2px solid #000',
  boxShadow: 24,
  p: 4,
};

function Home() {
  const users = useSelector(state => state.users);
  const marketplaces = useSelector(state => state.marketplaces);
  const dispatch = useDispatch();
  const navigate = useNavigate();

  const [advertForClone, setAdvertForClone] = useState({});
  const [advertForSold, setAdvertForSold] = useState({});

  const openAdvertForm = (e, advert) => {
    setAdvertForClone(advert)
    return setOpen(true);
  }
  const closeAdvertForm = () => setOpen(false);
  const [open, setOpen] = useState(false);
  const [numberOfAdverts, setNumberOfAdverts] = React.useState(1);
  const [openCompleteDialog, setOpenCompleteDialog] = useState(false);
  const [filter, setFilter] = React.useState(['OLX', 'AutoRIA']);
  const olx_access_token = !!marketplaces?.olx_access_token
  const ria_access_token = !!marketplaces?.ria_access_token
  const token = !!users?.token;
  const {t} = Tr();

  useEffect(() => {
    if (olx_access_token && ria_access_token) {
      dispatch(getAdverts({
        data: {
          olx_access_token: marketplaces?.olx_access_token,
          ria_access_token: marketplaces?.ria_access_token,
          ria_user_id: marketplaces?.ria_user_id,
          olxEnabled: true,
          autoriaEnabled: true
        }
      })), []);
    } else if (olx_access_token) {
      dispatch(getAdverts({
        data: {

```

					КПІ.ІП-8233.045440.03.13	Арк.
Змін.	Арк.	№ докум.	Підп.	Дата.		23

```

        olx_access_token: marketplaces?.olx_access_token,
        olxEnabled: true
    }
  }, []);
} else if (ria_access_token) {
  dispatch(getAdverts({
    data: {
      ria_access_token: marketplaces?.ria_access_token,
      ria_user_id: marketplaces?.ria_user_id,
      autoriaEnabled: true
    }
  })), []);
}
},[dispatch]);

const handleChangeFilter = (event) => {
  setFilter(event.target.value);
};

const handleSearch = (event, newValues) => {
  navigate(`#${newValues.id}`);
};
const onChangeNumberOfAdverts = (e) => setNumberOfAdverts(e.target.value);

const hundred = [...Array(100).keys()].map(foo => foo + 1);

const handleMarkAsSold = (e) => {
  e.preventDefault();
  dispatch(markAsSold({
    data: {
      advert: advertForSold,
      cnt: numberOfAdverts
    },
    token: users.token,
    onClose: handleCloseCompleteDialog
  })));
}

console.log({marketplaces})

const handleOpenCompleteDialog = (e, advert) => {
  setAdvertForSold(advert)
  setOpenCompleteDialog(true);
};

const handleCloseCompleteDialog = () => {
  setOpenCompleteDialog(false);
};

const completeDialog = (
  <Dialog
    open={openCompleteDialog}
    onClose={handleCloseCompleteDialog}
    aria-labelledby="alert-dialog-title"
    aria-describedby="alert-dialog-description"
  >
    <DialogTitle id="alert-dialog-title">

```

```

        {"Are you sure you want to mark the product as sold?"}
    </DialogTitle>
    <DialogContent>
        <DialogContentText id="alert-dialog-description">
            {t("This action affects the general statistics which cannot be changed.")}
            <br/>
            {t("Please select how many products did you sell.")}
        </DialogContentText>
    </DialogContent>

    <Box textAlign={"center"}>
        <InputLabel id="demo-simple-select-standard-label">{t("Number Of
Products")}</InputLabel>
        <Select
            labelId="demo-simple-select-standard-label"
            id="demo-simple-select-standard"
            value={numberOfAdverts}
            onChange={onChangeNumberOfAdverts}
            label="Age"
            sx={{width: '50%'}}
        >
            {hundred.map(el =>
                <MenuItem key={el} value={el}>{el}</MenuItem>
            )}
        </Select>
    </Box>
    <DialogActions>
        <Button onClick={handleCloseCompleteDialog}>Cancel</Button>
        <Button onClick={handleMarkAsSold} autoFocus>
            Yes
        </Button>
    </DialogActions>
</Dialog>
)

if (token) {
    return (
        <Box sx={{marginLeft: 30}}>
            <Box display={'flex'} >
                <FormControl style={{width: 150, marginTop: 10, marginLeft: 20, textAlign: "center"}}>
                    <InputLabel id="demo-multiple-name-label">Filter</InputLabel>
                    <Select
                        labelId="demo-multiple-name-label"
                        id="demo-multiple-name"
                        multiple
                        value={filter}
                        onChange={handleChangeFilter}
                        input={<OutlinedInput label="Filter" />}
                    >
                        { ['OLX', 'AutoRIA'].map((name) => (
                            <MenuItem
                                key={name}
                                value={name}
                            >
                                {name}
                            </MenuItem>
                        ))}
                    </Select>
                </Box>
            </Box>
        </Box>
    )
}

```

```

</Select>
</FormControl>
<Box style={{width: 150, marginTop: 10, marginLeft: 20, textAlign: "center"}}>
  <Autocomplete
    disablePortal
    id="combo-box-demo"
    options={marketplaces?.adverts}
    getOptionLabel={(option) => option.title}
    onChange={handleSearch}
    sx={{ width: 300 }}
    renderInput={(params) => <TextField {...params} label="Find one ..." />}
  />
</Box>
</Box>

{marketplaces?.adverts?.filter(el => filter.includes(el.source)).map((advert) =>
  <a name={advert.id} key={advert.id}>
    <Card sx={{ width: 320, display: 'inline-block', float: 'left', margin: 2}}>
      <Box style={{position: 'relative'}}>
        <ButtonBase style={{width: "100%"}} onClick={() => {window.open(advert.url,
          "_blank")}}>
          <CardMedia
            component="img"
            height="180"
            image={advert.image}
            alt="green iguana"
          />
          <div style={{position: 'absolute', top: 5, right: 5, width: '20%'}}>
            <img src={advert.sourceURL} alt="" style={{width: '100%'}}/>
          </div>
        </ButtonBase>
      </Box>
      <CardContent>
        <Typography gutterBottom variant="h5" component="div">
          {advert.title}
        </Typography>
        <Typography variant="body2" color="text.secondary" style={{textAlign: "left"}}>
          {t("Source")}: {advert.source}
        </Typography>
        <Typography variant="body2" color="text.secondary" style={{textAlign: "left"}}>
          {t("Price")}: {advert.price} {advert.currency}
        </Typography>
        <Typography variant="body2" color="text.secondary" style={{textAlign: "left"}}>
          {t("Sold")}: {advert.soldCount} {t("item(s)") }
        </Typography>
        <Typography variant="body2" color="text.secondary" style={{textAlign: "left"}}>
          {t("Status")}: {advert.status}
        </Typography>
        <Typography variant="body2" color="text.secondary" style={{textAlign: "left",
          color: "green"}}>
          {t("Creation Date")}: {moment(advert.createdAt).format('L')}
        </Typography>
        <Typography variant="body2" color="text.secondary" style={{textAlign: "left",
          color: "red"}}>
          {t("Expiration Date")}: {moment(advert.expireDate).format('L')}
        </Typography>
      </CardContent>
    </a>
  )}

```

					КП.ІП-8233.045440.03.13	Арк.
Змін.	Арк.	№ докум.	Підп.	Дата.		26

```

        <Box style={{backgroundColor: '#e6e4e5', display: "flex", justifyContent:
"center", alignContent: "center", flexDirection: "row", lineHeight: 2, marginTop: 5}}>
        <span style={{verticalAlign: "middle", marginRight:
20}}>{t("Statistics")}</span>
        <span style={{marginRight:
5}}><RemoveRedEyeOutlinedIcon/>{advert.statistics.advert_views}</span>
        <span style={{marginRight:
5}}><FavoriteBorderOutlinedIcon/>{advert.statistics.users_observing}</span>
        <span style={{marginRight:
5}}><LocalPhoneOutlinedIcon/>{advert.statistics.phone_views}</span>
        </Box>
      </CardContent>

      <Box>
        <CardActions>
          <Button
            style={{float: "left", color: 'green'}}
            size="small"
            onClick={(e) => handleOpenCompleteDialog(e, advert)}
          >
            <CheckCircleOutlineRoundedIcon/>
          </Button>
          {completeDialog}

          <Button
            style={{float: "left", color: 'red'}}
            size="small"
            onClick={(e) => openAdvertForm(e, advert)}
          >
            <DeleteOutlineRoundedIcon/>
          </Button>

          <Button style={{float: "right", color: 'grey'}}
            onClick={()=>{window.open(advert.url, "_blank")}} size="small">{t("GO TO SOURCE")}</Button>
        </CardActions>
      </Box>
    </Card>
  </a>
)
</Box>
);
}

return (
  <Box>
    <Typography variant="h4" component="h2">
      {t("HOME")}
    </Typography>
    <Typography variant="h6" component="h2">
      {t("Please login to add trading platform")}
    </Typography>
  </Box>
);
}

export default Home;

```

```

import React, {useEffect, useState} from "react";
import {Box, Button, Typography, Modal, TextField, FormControl, InputLabel, Select, MenuItem} from
'@mui/material';
import {useTranslation} from 'react-i18next'
import {useSearchParams, useNavigate} from "react-router-dom";
import {useDispatch, useSelector} from "react-redux";
import {createRiaSession, createSession, getAdverts, getSoldStatistics} from
"./../redux/modules/marketplaces";
import ArrowDropUpRoundedIcon from '@mui/icons-material/ArrowDropUpRounded';
import moment from 'moment';
import { Chart as ChartJS, ArcElement, Tooltip, Legend, RadialLinearScale, CategoryScale,
  LinearScale,
  PointElement,
  LineElement,
  Title,
  BarElement
} from 'chart.js';

import { Doughnut, Line, Pie, Bar } from 'react-chartjs-2';
import './styles.css'

```

```

ChartJS.register(RadialLinearScale, ArcElement, Tooltip, Legend, CategoryScale, LinearScale,
  PointElement, LineElement, Title, BarElement);

```

```

const Tr = useTranslation;
const getDaysArray = function(start, end) {
  const arr = [];
  for(const dt = new Date(start); dt<=new Date(end); dt.setDate(dt.getDate()+1)){
    arr.push(moment(new Date(dt)).format('L'))
  }
  return arr;
};

```

```

function daysInMonth(month) {
  if (month === -1){
    month = new Date().getMonth();
  }
  const year = new Date().getFullYear()
  return new Date(year, month, 0).getDate();
}

```

```

function getRandom(min, max) {
  return Math.round(Math.random() * (max - min) + min);
}

```

```

function changeDays(d, p, marketplaces = null) {
  let tmp1 = [];
  let tmp2 = [];
  let tmp3 = [];
  for (let i = 0; i <= d; i++){
    if (p){
      let dateOfProduct = new Date(p.createdAt)
      let date = new Date(Date.now() - i * 24 * 60 * 60 * 1000);
      if (date >= dateOfProduct){
        let to = d-i;
        tmp1.push(getRandom(to, to*2));
      }
    }
  }
}

```

					КПІ.ІП-8233.045440.03.13	Арк.
						28
Змін.	Арк.	№ докум.	Підп.	Дата.		

```

        tmp2.push(getRandom(to/5, to/2));
        tmp3.push(getRandom(to/5, to/2));
    }
    else {
        tmp1.unshift(0);
        tmp2.unshift(0);
        tmp3.unshift(0);
    }
}
else if (marketplaces?.adverts){
    let minDate = new Date();
    marketplaces?.adverts?.map(advert => {

        let newDate = new Date(advert.createdAt)
        if (newDate < minDate){
            minDate = newDate;
        }
    })
    let strMinDate = {
        createdAt: `${minDate}`
    }
    return changeDays(d, strMinDate);
}
}
return {tmp1, tmp2, tmp3}
}

function Statistics() {
    const navigate = useNavigate();
    const dispatch = useDispatch();
    const marketplaces = useSelector(state => state.marketplaces);
    const users = useSelector(state => state.users);

    const olx_access_token = !!marketplaces?.olx_access_token
    const ria_access_token = !!marketplaces?.ria_access_token

    const [product, setProduct] = React.useState(false);

    const [days, setDays] = React.useState(6);

    let daysData;
    if (marketplaces){
        daysData = changeDays(days, product, marketplaces);
    }
    let dataViews = daysData.tmp1;
    let dataObserving = daysData.tmp2;
    let dataViewsPhone = daysData.tmp3;

    const handleChange = (event) => {
        setProduct(event.target.value);
        const {tmp1, tmp2, tmp3} = changeDays(days, event.target.value, marketplaces);
        dataViews = tmp1;
        dataObserving = tmp2;
        dataViewsPhone = tmp3;
    };

    const handleChangeDays = (event) => {

```

					КПІ.ІП-8233.045440.03.13	Арк.
						29
Змін.	Арк.	№ докум.	Підп.	Дата.		


```

setDays(event.target.value);
const d = event.target.value
const {tmp1, tmp2, tmp3} = changeDays(d, product, marketplaces)
dataViews = tmp1;
dataObserving = tmp2;
dataViewsPhone = tmp3;
};

console.log(marketplaces);

useEffect(() => {
  dispatch(getSoldStatistics({
    token: users.token,
  }), []);
  if (olx_access_token && ria_access_token) {
    dispatch(getAdverts({
      data: {
        olx_access_token: marketplaces?.olx_access_token,
        ria_access_token: marketplaces?.ria_access_token,
        ria_user_id: marketplaces?.ria_user_id,
        olxEnabled: true,
        autoriaEnabled: true
      }
    }), []);
  } else if (olx_access_token) {
    dispatch(getAdverts({
      data: {
        olx_access_token: marketplaces?.olx_access_token,
        olxEnabled: true
      }
    }), []);
  } else if (ria_access_token) {
    dispatch(getAdverts({
      data: {
        ria_access_token: marketplaces?.ria_access_token,
        ria_user_id: marketplaces?.ria_user_id,
        autoriaEnabled: true
      }
    }), []);
  }
}, [dispatch]);

const {t} = Tr();

const lineOptions = {
  responsive: true,
  plugins: {
    legend: {
      position: 'top',
    },
    title: {
      display: true,
      text: t('Active Statistics Line Graphic'),
    },
  },
};

```

```

if (product) console.log(getDaysArray(new Date(product.createdAt), new Date()));

const lineData = {
  labels: getDaysArray((new Date()).setDate((new Date()).getDate() - days), new Date()),
  datasets: [
    {
      label: t('Number Of Views'),
      data: dataViews,
      fill: false,
      borderColor: 'rgb(75, 192, 192)',
      tension: 0.1
    },
    {
      label: t('Number Of Observing'),
      data: dataObserving,
      borderColor: 'rgb(255, 99, 132)',
      backgroundColor: 'rgba(255, 99, 132, 0.5)',
    },
    {
      label: t("Number Of Phone Views"),
      data: dataViewsPhone,
      borderColor: 'rgb(53, 162, 235)',
      backgroundColor: 'rgba(53, 162, 235, 0.5)',
    },
  ],
};

const sourceData = {
  labels: [ 'OLX', 'AutoRIA'],
  datasets: [
    {
      label: 'number of ads',
      data: [marketplaces?.sourceCount?.OLX, marketplaces?.sourceCount?.AutoRIA],
      backgroundColor: [
        'rgba(255, 99, 132, 0.2)',
        'rgba(54, 162, 235, 0.2)',
      ],
      borderColor: [
        'rgba(255, 99, 132, 1)',
        'rgba(54, 162, 235, 1)',
      ],
      borderWidth: 1,
    },
  ],
};

const profitOptions = {
  aspectRatio: 3,
  plugins: {
    legend: {
      position: 'top',
    },
    title: {
      display: true,
      text: t("Top Profitable Products"),
    },
  },
};

```

					КПІ.ІП-8233.045440.03.13	Арк.
Змін.	Арк.	№ докум.	Підп.	Дата.		31

```

    }
  };

  const profitData = {
    labels: marketplaces?.soldStatistics?.products?.map(el => el?.title),
    datasets: [
      {
        label: t("Profit"),
        data: marketplaces?.soldStatistics?.products?.map(el => el?.total),
        backgroundColor: 'rgba(84, 176, 77, 0.5)',
      },
    ],
  };

  const viewsData = {
    labels: marketplaces?.adverts?.map(el => `${el?.title} (${el.source})`),
    datasets: [
      {
        label: t("Number of views"),
        data: marketplaces?.adverts?.map(el => el.statistics?.advert_views),
        backgroundColor: [
          'rgba(255, 99, 132, 0.2)',
          'rgba(54, 162, 235, 0.2)',
          'rgba(255, 206, 86, 0.2)',
          'rgba(75, 192, 192, 0.2)',
          'rgba(153, 102, 255, 0.2)',
          'rgba(255, 159, 64, 0.2)',
        ],
        borderColor: [
          'rgba(255, 99, 132, 1)',
          'rgba(54, 162, 235, 1)',
          'rgba(255, 206, 86, 1)',
          'rgba(75, 192, 192, 1)',
          'rgba(153, 102, 255, 1)',
          'rgba(255, 159, 64, 1)',
        ],
        borderWidth: 1,
      },
    ],
  };

  const observingData = {
    labels: marketplaces?.adverts?.map(el => `${el?.title} (${el.source})`),
    datasets: [
      {
        label: 'number of observing',
        data: marketplaces?.adverts?.map(el => el.statistics?.users_observing),
        backgroundColor: [
          'rgba(255, 99, 132, 0.2)',
          'rgba(54, 162, 235, 0.2)',
          'rgba(255, 206, 86, 0.2)',
          'rgba(75, 192, 192, 0.2)',
          'rgba(153, 102, 255, 0.2)',
          'rgba(255, 159, 64, 0.2)',
        ],
        borderColor: [

```

```

        'rgba(255, 99, 132, 1)',
        'rgba(54, 162, 235, 1)',
        'rgba(255, 206, 86, 1)',
        'rgba(75, 192, 192, 1)',
        'rgba(153, 102, 255, 1)',
        'rgba(255, 159, 64, 1)',
    ],
    borderWidth: 1,
  },
],
};

const phoneViewsData = {
  labels: marketplaces?.adverts?.map(el => `${el?.title} (${el.source})`),
  datasets: [
    {
      label: 'number of views',
      data: marketplaces?.adverts?.map(el => el.statistics?.phone_views),
      backgroundColor: [
        'rgba(255, 99, 132, 0.2)',
        'rgba(54, 162, 235, 0.2)',
        'rgba(255, 206, 86, 0.2)',
        'rgba(75, 192, 192, 0.2)',
        'rgba(153, 102, 255, 0.2)',
        'rgba(255, 159, 64, 0.2)',
      ],
      borderColor: [
        'rgba(255, 99, 132, 1)',
        'rgba(54, 162, 235, 1)',
        'rgba(255, 206, 86, 1)',
        'rgba(75, 192, 192, 1)',
        'rgba(153, 102, 255, 1)',
        'rgba(255, 159, 64, 1)',
      ],
      borderWidth: 1,
    },
  ],
};

```

```

return (
  <Box sx={{marginLeft: 30}}>
    <Typography variant="h4" component="h1" sx={{m: 2}}>
      {t("Statistics")}
    </Typography>
    <Box sx={{display: 'flex'}}>
      <Box sx={{flex: 1, marginLeft: 10}}>
        <Box sx={{height: 140}} className='hvr-box-shadow-outset'>
          <div style={{width: '40%', height: 30, textAlign: "center", display: 'inline-block',
marginTop: 15, backgroundColor: '#54B04D', borderRadius: 5}}>
            <p style={{color: 'white', lineHeight: 0}}>
              {t("Profit")}
            </p>
          </div>

          <p style={{color: '#54B04D'}}>
            {marketplaces?.soldStatistics?.statistics?.totalPrice} $

```

```

</p>

<div style={{}}>
  <ArrowDropUpRoundedIcon sx={{color: '#54B04D', verticalAlign: 'middle'}}/>
  <span style={{verticalAlign: 'middle', fontSize: 12}}>+
{marketplaces?.soldStatistics?.forPeriod?.totalPrice} $ {"in this month"}</span>
</div>

</Box>

<Box sx={{height: 140}} className='hvr-box-shadow-outset'>
  <div style={{width: '40%', height: 30, textAlign: "center", display: 'inline-block',
marginTop: 15, backgroundColor: 'orange', borderRadius: 5}}>
    <p style={{color: 'white', lineHeight: 0}}>
      {"Sold"}
    </p>
  </div>

  <p style={{color: 'orange'}}>
    {marketplaces?.soldStatistics?.statistics?.totalCount} {"item(s)}
  </p>

  <div style={{}}>
    <ArrowDropUpRoundedIcon sx={{color: 'orange', verticalAlign: 'middle'}}/>
    <span style={{verticalAlign: 'middle', fontSize: 12}}>+
{marketplaces?.soldStatistics?.forPeriod?.totalCount} {"item(s)} {"in this month"}</span>
  </div>
</Box>

</Box>

<Box sx={{flex: 3, marginLeft: 10, marginRight: 10}}>
  <Bar options={profitOptions} data={profitData} style={{}} />
</Box>

</Box>

<Box>
  <div style={{display: "flex"}}>
    <Box style={{display: 'inline-block', width: '25%', flex: 1}}>
      <Typography variant="body1" sx={{m: 2}}>
        {"Number Of Ads"}
      </Typography>
      <Doughnut data={sourceData} />
    </Box>

    <Box style={{display: 'inline-block', flex: 3}}>
      <Typography variant="h6" component="h1" sx={{m: 2}}>
        {"Active Statistics"}
      </Typography>
      <Box style={{display: 'inline-block', width: '25%'}}>
        <Typography variant="body1" sx={{m: 2}}>
          {"Number Of Views"}
        </Typography>
        <Pie data={viewsData} />
      </Box>
    </Box>
  </div>
</Box>

```

```

<Box style={{display: 'inline-block', width: '25%'}}>
  <Typography variant="body1" sx={{m: 2}}>
    {t("Number Of Observing")}
  </Typography>
  <Pie data={observingData} />
</Box>

<Box style={{display: 'inline-block', width: '25%'}}>
  <Typography variant="body1" sx={{m: 2}}>
    {t("Number Of Phone Views")}
  </Typography>
  <Pie data={phoneViewsData} />
</Box>
</div>

<Box style={{textAlign: 'center', margin: 10}}>
  <Box display={flex} style={{marginTop: 40}}>
    <FormControl variant="standard" sx={{ m: 1, minWidth: 130 }}>
      <InputLabel>{t("Select Product")}</InputLabel>
      <Select
        value={product}
        onChange={handleChange}
        label={t("Select Product")}
        style={{width: "100%"}}
      >
        <MenuItem value="">
          <em>None</em>
        </MenuItem>

        {marketplaces?.adverts?.map(advert =>
          <MenuItem value={advert} key={advert.id} >{advert.title}</MenuItem>
        )}
      </Select>
    </FormControl>
    <FormControl variant="standard" sx={{ m: 1, minWidth: 130 }}>
      <InputLabel>{t("Select time")}</InputLabel>
      <Select
        value={days}
        onChange={handleChangeDays}
        lable="Select time"
      >
        <MenuItem value={6}>
          <em>{t("week")}</em>
        </MenuItem>
        <MenuItem value={daysInMonth(-1)}>
          <em>{t("month")}</em>
        </MenuItem>
        <MenuItem value={365}>
          <em>{t("year")}</em>
        </MenuItem>
      </Select>
    </FormControl>
  </Box>
  <Line options={lineOptions} data={lineData} />
</Box>
</Box>

```

					КПІ.ІП-8233.045440.03.13	Арк.
Змін.	Арк.	№ докум.	Підп.	Дата.		35

```

    </Box>
  );
}

export default Statistics;

import * as React from 'react';
import {
  Button,
  Avatar,
  ButtonBase,
  Drawer,
  List,
  Divider,
  ListItem,
  ListItemButton,
  ListItemIcon,
  ListItemText,
} from "@mui/material";
import {styleToolbar} from "../styles/main";
import {Link} from "react-router-dom"
import {
  Box,
  Fab,
  AppBar,
  Tooltip,
  Toolbar,
  useScrollTrigger,
  Slide,
} from "@mui/material";
import {AttachMoney} from '@mui/icons-material';
import {Home, Settings, DisplaySettings} from "@mui/icons-material"
import * as rr from "react-redux";
import * as rd from "react-router-dom";
import config from "../config/config";
import {parseToken} from '../utils/parseToken';
import Lg from './lgSelector'
import {useTranslation} from 'react-i18next'
import {AlertMessage} from "../utils/alert";
import PropTypes from 'prop-types';
import KeyboardArrowUpIcon from '@mui/icons-material/KeyboardArrowUp';
import Zoom from '@mui/material/Zoom';
const Tr = useTranslation;

export function ScrollTop(props) {
  const { children } = props;
  const trigger = useScrollTrigger({
    disableHysteresis: true,
    threshold: 100,
  });

  const handleClick = (event) => {
    event.view.scrollTo({
      top: 0,
      behavior: 'smooth'
    });
  }

```

```

    });
  };

  return (
    <Zoom in={trigger}>
      <Box
        onClick={handleClick}
        role="presentation"
        sx={{ position: 'fixed', bottom: 16, right: 16 }}
      >
        {children}
      </Box>
    </Zoom>
  );
}

ScrollTop.propTypes = {
  children: PropTypes.element.isRequired,
  window: PropTypes.func,
};

const drawerWidth = 240;

export default function ToolbarMain(props) {
  const { window } = props;
  const { t } = Tr();
  const users = rr.useSelector(state => state.users);
  const dispatch = rr.useDispatch();
  const navigate = rd.useNavigate();
  const decode = parseToken(users.token);
  const container = window !== undefined ? () => window().document.body : undefined;
  const [mobileOpen, setMobileOpen] = React.useState(false);

  const handleDrawerToggle = () => {
    setMobileOpen(!mobileOpen);
  };

  const register = () => {
    navigate('/register');
  }

  const settings = () => {
    navigate('/settings')
  }

  const home = () => {
    navigate('/')
  }

  const tradingPlatforms = () => {
    navigate('/trading-platform')
  }

  const statistics = () => {
    navigate('/statistics')
  }
}

```



```

const drawer = (
  <div>
    <Toolbar />
    <Divider />
    <List>
      {[{
        text: t("Home"),
        func: home
      }, {
        text: t("Statistics"),
        func: statistics
      }, {
        text: t("Trading Platforms"),
        func: tradingPlatforms
      }
      ]
      .map(({text, func}) => (
        <ListItem key={text} disablePadding>
          <ListItemButton onClick={func}>
            <ListItemIcon>
              <AttachMoney />
            </ListItemIcon>
            <ListItemText primary={text} />
          </ListItemButton>
        </ListItem>
      ))}
    </List>
    <Divider />
    <List>
      {[{text: t("Settings"), func: settings}].map(({text, func}) => (
        <ListItem key={text} disablePadding>
          <ListItemButton onClick={func}>
            <ListItemIcon>
              <DisplaySettings />
            </ListItemIcon>
            <ListItemText primary={text} />
          </ListItemButton>
        </ListItem>
      ))}
    </List>
  </div>
);

return (
  <div>
    <AppBar style={styleToolbar.toolbar}>
      <Toolbar>
        <Box display={'flex'} style={{flexGrow: 7}}> </Box>
        {users.token &&
          <Box display={'flex'}>
            <Tooltip title="home" arrow>
              <Link to={'/'}>
                <Fab color="primary" size="small">
                  <Home />
                </Fab>
              </Link>
            </Box>
        }
      </Toolbar>
    </AppBar>
  </div>
);

```

```

    </Tooltip>
  </Box>
}
{!users.token && <Lg/>}
{!users.token &&
  <div>
    <Link style={styleToolbar.Link} to="/login">{t("sing in")}</Link>
    <Button
      style={styleToolbar.button}
      onClick={register}
      variant='contained' color='primary'
    >
      {t("register")}
    </Button>
  </div>
}
</Toolbar>
</AppBar>
<Toolbar />
{users.token &&
  <div>
    <Drawer
      container={container}
      variant="temporary"
      open={mobileOpen}
      onClose={handleDrawerToggle}
      ModalProps={{
        keepMounted: true, // Better open performance on mobile.
      }}
      sx={{
        display: {sm: 'block', md: 'none'},
        '& .MuiDrawer-paper': {boxSizing: 'border-box', width: drawerWidth},
      }}
    >
      {drawer}
    </Drawer>
    <Drawer
      variant="permanent"
      sx={{
        display: {xs: 'none', md: 'block'},
        '& .MuiDrawer-paper': {boxSizing: 'border-box', width: drawerWidth},
      }}
      open
    >
      {drawer}
    </Drawer>
  </div>
}
<Box display={ 'block' } style={{width: 250, margin: 15, right: 15, position: "absolute"}}>
  <AlertMessage error={users.error} success={users.success}/>
</Box>
<ScrollTop {...props}>
  <Fab color="secondary" size="small" aria-label="scroll back to top">
    <KeyboardArrowUpIcon />
  </Fab>
</ScrollTop>
</div>

```

```
);  
}
```

```
import React from 'react'  
import i18next from 'i18next'  
import MenuItem from '@mui/material/MenuItem';  
import FormControl from '@mui/material/FormControl';  
import Select from '@mui/material/Select';  
import {languages} from "../../config/trOptions";  
import * as r from "react";  
  
export default function lg() {  
  const currentLanguageCode = localStorage.getItem('i18nextLng') || 'en'  
  
  const [lg, setLg] = r.useState(currentLanguageCode);  
  
  const handleChange = (event) => {  
    setLg(event.target.value);  
    i18next.changeLanguage(event.target.value)  
  };  
  
  return (  
    <div className="container">  
      <FormControl variant="standard" sx={{ m: 1 }} >  
        <Select  
          style={{color: "#cbac4a"}}  
          value={lg}  
          onChange={handleChange}  
          displayEmpty  
          inputProps={{ 'aria-label': 'Without label' }}  
        >  
          {languages.map(e => (  
            <MenuItem value={e.code} key={e.code}>  
              {e.name} {e.country_flag}  
            </MenuItem>  
          ))}  
        </Select>  
      </FormControl>  
    </div>  
  )  
}
```

Факультет інформатики та обчислювальної техніки
Кафедра інформатики та програмної інженерії

“ЗАТВЕРДЖЕНО”

Завідувач кафедри

_____ Едуард ЖАРІКОВ

“ ____ ” _____ 2022 р.

**ПРОГРАМНЕ ЗАБЕЗПЕЧЕННЯ ДЛЯ ПІДТРИМКИ ТА АНАЛІТИКИ
ДЛЯ ПРОДАВЦІВ НА ТОРГОВЕЛЬНИХ МАЙДАНЧИКАХ**

Програма та методика тестування

КП.ІП-8233.045440.04.51

“ПОГОДЖЕНО”

Керівник проекту:

_____ Олексій ФІНОГЕНОВ

Нормоконтроль:

_____ Катерина ЛІЩУК

Виконавець:

_____ Олександр ФІЛАТОВ

Київ – 2022

ЗМІСТ

1	ОБ'ЄКТ ВИПРОБУВАНЬ.....	2
2	МЕТА ТЕСТУВАННЯ.....	3
3	МЕТОДИ ТЕСТУВАННЯ.....	4
4	ЗАСОБИ ТА ПОРЯДОК ТЕСТУВАННЯ	5

					КПІ.ІП-8233.045440.04.51	Арк.
Змін.	Арк.	№ докум.	Підп.	Дата.		1

1 ОБ'ЄКТ ВИПРОБУВАНЬ

Об'єктами випробувань є:

- вебдодаток;
- серверна частина програмного забезпечення.

					КПІ.ІП-8233.045440.04.51	Арк.
Змін.	Арк.	№ докум.	Підп.	Дата.		2

2 МЕТА ТЕСТУВАННЯ

Метою тестування є наступне:

- перевірка правильності роботи сервеної частини відповідно до функціональних вимог;
- перевірка правильності роботи вебдодатку;
- знаходження проблем та недоліків з метою їх усунення;
- перевірка зручності графічного інтерфейсу.

					КПІ.ІП-8233.045440.04.51	Арк.
Змін.	Арк.	№ докум.	Підп.	Дата.		3

3 МЕТОДИ ТЕСТУВАННЯ

Методи тестування можна поділити на наступні рівні:

- модульне тестування. Виконується розробником на етапі кодування програми. Модульні тести ізолюють частину коду та перевіряють його працездатність. Одиницею для вимірювання може бути окрема функція, метод, процедура, модуль чи об'єкт. Тести реалізуються за допомогою стороннього додатку AVA;
- інтеграційне тестування. Для інтеграційного тестування використовуються компоненти, що вже перевірені за допомогою модульного тестування, які групуються у множини. Дані множини перевіряються відповідно до плану тестування, складеного для них, а об'єднуються вони через свої інтерфейси. Тести реалізуються за допомогою стороннього додатку AVA;
- UI тестування. Цей процес передбачає імітацію дій користувача — кліки на кнопки, переходи по посиланнях, та інші дії подібного плану. Таким чином перевіряється коректність роботи, взаємодія компонентів один з одним та зручність інтерфейсу в цілому.

Тести вважаються пройденими у випадку у випадку не знаходження жодних помилок на усіх рівнях тестування. Також мають бути виконані усі вимоги зазначені у технічному завданні.

					КПІ.ІП-8233.045440.04.51	Арк.
Змін.	Арк.	№ докум.	Підп.	Дата.		4

4 ЗАСОБИ ТА ПОРЯДОК ТЕСТУВАННЯ

Для того, щоб перевірити працездатність додатку, необхідно провести наступні тестування:

- UI тестування вебдодатку на пристроях з різним розміром, співвідношенням сторін екрану.
- тестування функціональних вимог будь-яким методом;
- тестування нефункціональних вимог будь-яким методом.

					КПІ.ІП-8233.045440.04.51	Арк.
Змін.	Арк.	№ докум.	Підп.	Дата.		5

Факультет інформатики та обчислювальної техніки
Кафедра інформатики та програмної інженерії

“ЗАТВЕРДЖЕНО”

Завідувач кафедри

_____ Едуард ЖАРІКОВ

“ ____ ” _____ 2022 р.

**ПРОГРАМНЕ ЗАБЕЗПЕЧЕННЯ ДЛЯ ПІДТРИМКИ ТА АНАЛІТИКИ
ДЛЯ ПРОДАВЦІВ НА ТОРГОВЕЛЬНИХ МАЙДАНЧИКАХ**

Керівництво користувача

КПІ.ІП-8233.045440.05.34

“ПОГОДЖЕНО”

Керівник проекту:

_____ Олексій ФІНОГЕНОВ

Нормоконтроль:

_____ Катерина ЛІЩУК

Виконавець:

_____ Олександр ФІЛАТОВ

Київ – 2022

ЗМІСТ

1	ЗАГАЛЬНІ ВІДОМОСТІ	2
2	ПІДГОТОВКА ДО РОБОТИ.....	3
2.1	СИСТЕМНІ ВИМОГИ ДЛЯ КОРЕКТНОЇ РОБОТИ	3
2.2	ЗАВАНТАЖЕННЯ ЗАСТОСУНКУ	3
2.3	ПЕРЕВІРКА КОРЕКТНОЇ РОБОТИ	3
3	РОБОТА ІЗ ЗАСТОСУНКОМ	4

					КПІ.ІП-8233.045440.05.34	Арк.
Змін.	Арк.	№ докум.	Підп.	Дата.		1

1 ЗАГАЛЬНІ ВІДОМОСТІ

Програмна забезпечення було розроблено виключно для продавців які використовують один чи більше торговельних майданчиків.

Отже, не має необхідності по черзі заходити в особисті кабінети на всіх сайтах торговельних майданчиків, щоб побачити результати торгівлі. Це все з легкістю можна буде побачити на графіках статистик, які включають прибуток по всім каналам продаж і по кожному в окремої можна подивитися в одному місці. А якщо потрібно, то автоматично оновлювати статуси на всіх товарах. Можна проаналізувати статистику продажів, прослідкувати за оборотами, оцінити рентабельність. Тобто це прискорить процес аналітики за оголошеннями та поліпшить комфорт використання.

					КПІ.ІП-8233.045440.05.34	Арк.
Змін.	Арк.	№ докум.	Підп.	Дата.		2

2 ПІДГОТОВКА ДО РОБОТИ

2.1 Системні вимоги для коректної роботи

Так як розроблений додаток кросплатформний, необхідно мати пристрій з виходом в інтернет. Найкращим с точки зору комфортності буде персональний комп'ютер.

2.2 Завантаження застосунку

Для користування вебдодатком найкращим варіантом буде завантаження одного з новітніх браузерів таких як Chrome. Підтримуються всі ОС у який є вихід в інтернет.

Для налаштуванні на персональному комп'ютері можна завантажити Chrome додаток. Підтримуються наступні ОС:

- Windows;
- Linux;
- MacOS.

2.3 Перевірка коректної роботи

Після переходу до ресурсу. Повинна відобразитися форма з реєстрацією. Якщо реєстрація та вхід пройшли успішно це означає що вебдодаток та серверна частина з СУБД працюють коректно.

					КПІ.ІП-8233.045440.05.34	Арк.
Змін.	Арк.	№ докум.	Підп.	Дата.		3

3 РОБОТА ІЗ ЗАСТОСУНКОМ

При запуску веб додатка пропонується авторизуватися або зареєструватися в додатку, як показано на рисунку 3.1.

Вхід

ДАЛІ

Рисунок 3.1 – Авторизація у вебдодатку

Для того, щоб приєднати торговельні майданчики необхідно перейти на вкладку «Trading Platforms» та вибрати для підключення бажаний сервіс, як показано на рисунку 3.2.

Виберіть Торговельний Майданчик



АВТОРИЗУВАТИСЯ ЧЕРЕЗ OLX



АВТОРИЗУВАТИСЯ ЧЕРЕЗ AUTORIA

Рисунок 3.2 – Сторінка для підключення торговельних майданчиків

При підключенні сервісу OLX потрібно надати дозвіл, як показано на
рисунку 3.3.

Додаток Analytic Subsystem

Дозволити доступ до вашого аккаунту?

alexfilatov380 (alexfilatov380@gmail.com)

Додатку будуть доступні:

читання даних профілю, оголошень, повідомлень
редагування даних профілю, оголошень,
повідомлень

Ви можете змінити налаштування доступу в вашому
Профілі.

Вибравши "Дозволити", ви надаєте програмі Analytic
Subsystem доступ до свого облікового запису OLX і
дозволяєте їй діяти від вашого імені відповідно до Угоди
користувача сервісів OLX та умов використання додатку.

ДОЗВОЛИТИ

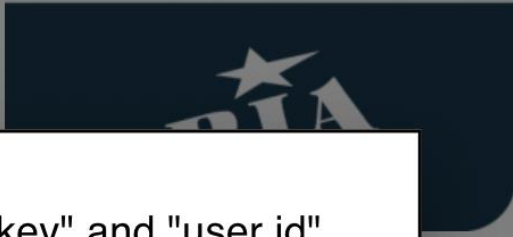
ВІДМІНА

Рисунок 3.3 – Авторизація через сервіс OLX

При підключенні сервісу AutoRia необхідно ввести свої унікальні секрети,
які можна знайти у своєму обліковому записі, та ввести їх у форму (рис. 3.4).

					КПІ.ІП-8233.045440.05.34	Арк.
Змін.	Арк.	№ докум.	Підп.	Дата.		6

Виберіть Торговельний Майданчик



ABTOPI JTORIA

Please provide your "api key" and "user id"

api key *

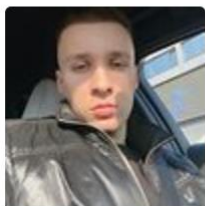
user id *

SUBMIT

Рисунок 3.4 – Авторизація через сервіс AutoRia

На рисунку 3.5 зображена сторінка профілю AutoRia де можна знайти секрети для доступу до товарів.

Мой профиль



Email	alexfilatov380@gmail.com
Ваш ID (user_id)	11267851
Api Key	9T6v4q... Показати
Зареєстрований	2022-05-18 11:15:01

[Редагувати](#)[Змінити пароль](#)

Рисунок 3.5 – Сторінка з ключами доступу до AutoRia

Після підключення торговельних майданчиків користувач зможе побачити та відфільтрувати всі свої продукти на головній сторінці вебдодатка (рис. 3.6).



Iphone 11 256gb black

Торговельний Майданчик: OLX

Ціна: 1000 USD

Продано: 12 шт.

Статус: active

Дата Створення: 05/14/2022

Термін Придатності: 06/13/2022

Статистика: 10 0 0



ПЕРЕЙТИ ДО
РЕСУРСУ



Honda CR-V 2007

Торговельний Майданчик: AutoRIA

Ціна: 10000 USD

Продано: 1 шт.

Статус: active

Дата Створення: 05/18/2022

Термін Придатності: 06/18/2022

Статистика: 353 0 2



ПЕРЕЙТИ ДО
РЕСУРСУ

Рисунок 3.6 – Демонстрація карток товарів

На сторінці статистики користувачу буде продемонстрований рейтинг товарів (рис. 3.7) та загальний прибуток (рис. 3.8).

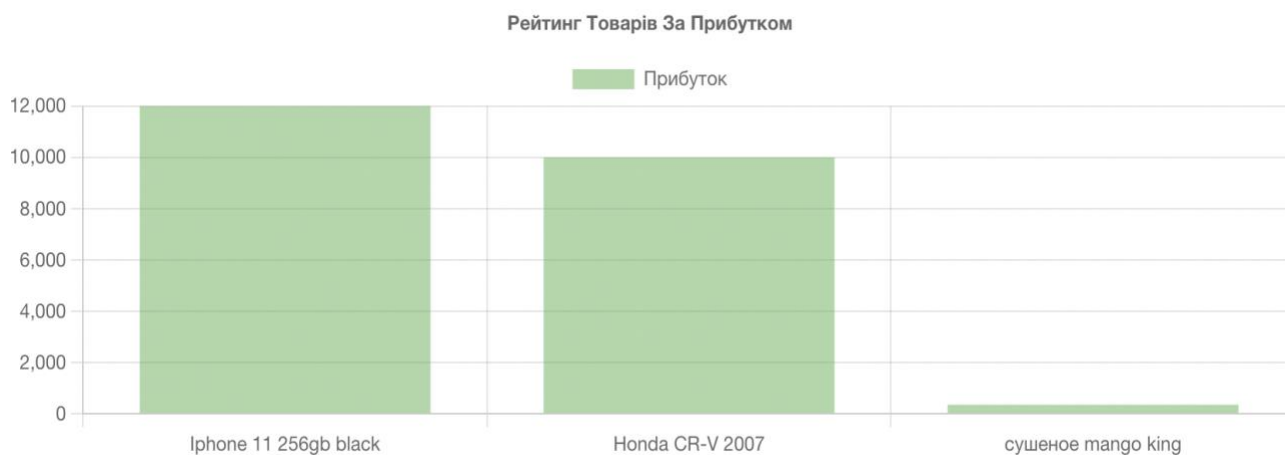


Рисунок 3.7 – Рейтинг товарів

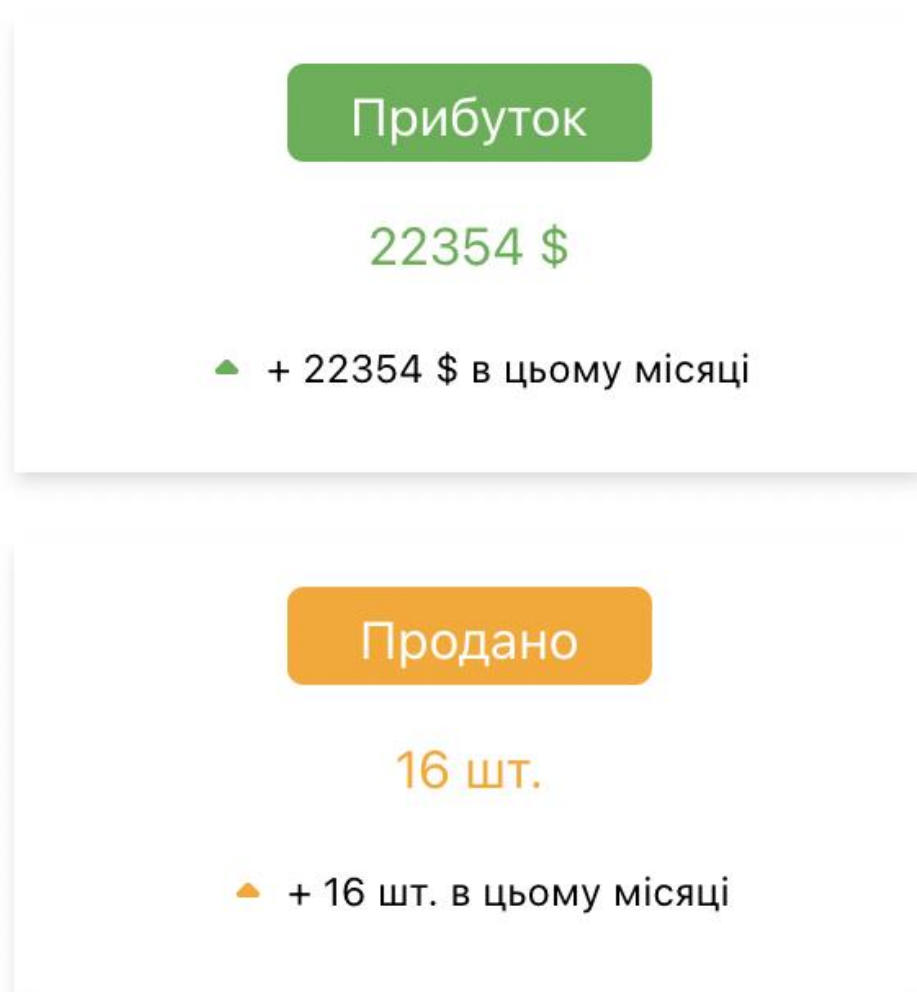


Рисунок 3.8 – Рейтинг товарів

Також користувач зможе скористатися на сторінці статистики лінійний графік по кожному з товарів в якому він зацікавлений попередньо вказавши діапазон (тиждень, місяць або рік), як показано на рисунку 3.9.

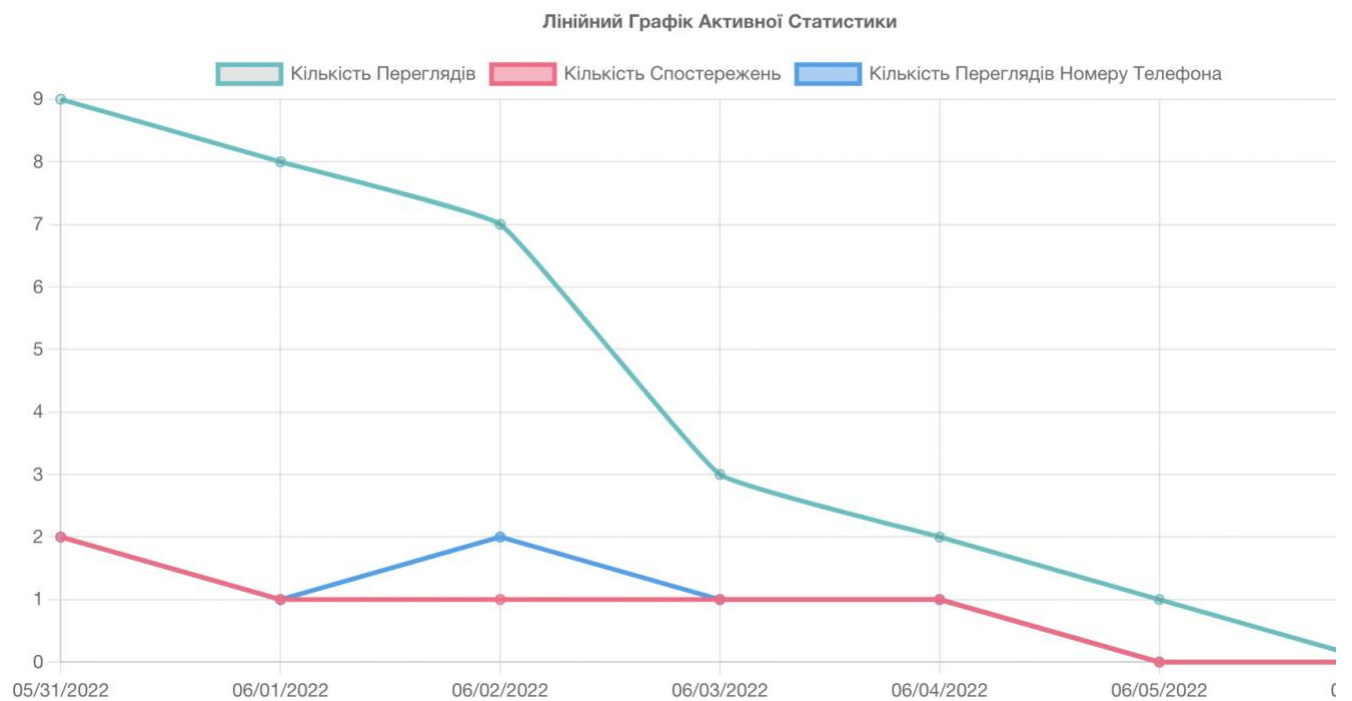


Рисунок 3.9 – Лінійний графік активності товару

Користувач має можливість переглядати активну статистику за трьома пунктами це кількість переглядів (рис. 3.10), кількість спостережень та кількість переглядів номеру телефона.

Кількість Переглядів

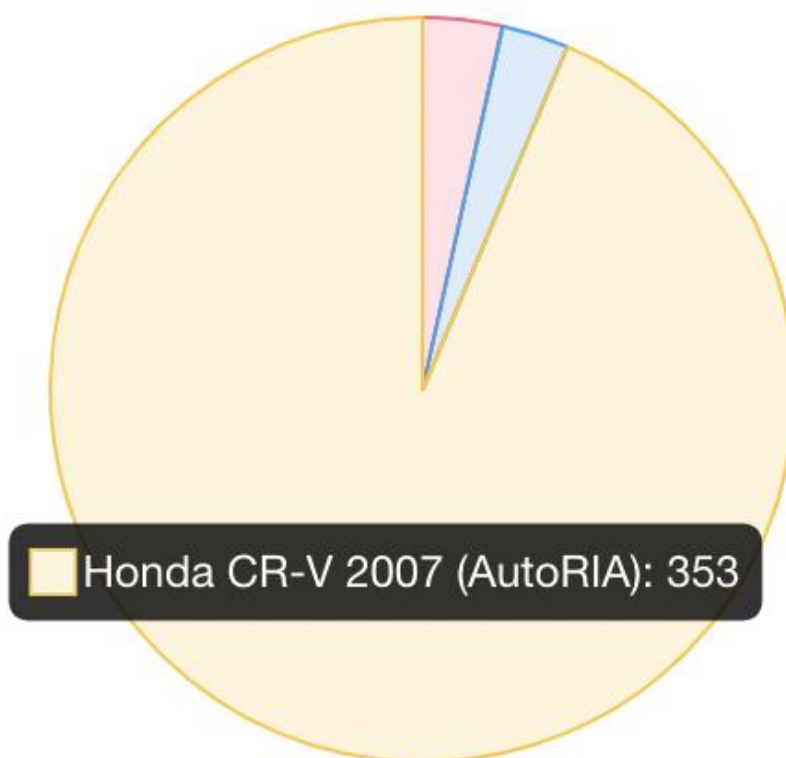
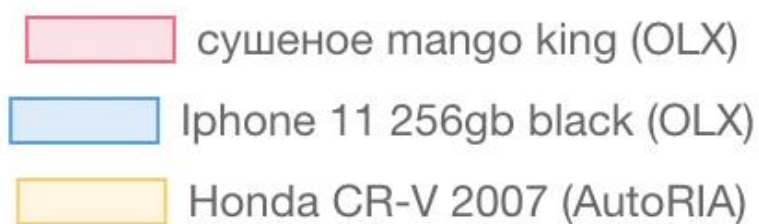


Рисунок 3.10 – Діаграма за кількістю переглядів

Щоб переглянути власний профіль треба натиснути на лівій боковій панелі кнопку «Налаштування» (є можливість зміни мови), рисунок 3.11.

Мова

Українська  ▼



Персональна Інформація

email

alexfilatov380@gmail.com

password

ВИЙТИ

Рисунок 3.11 – Профіль користувача